



File BT Kiến trúc Ảo m Ảy t Ảnh

kiến trúc máy tính (Học viện Công nghệ Bưu chính Viễn thông)



Scan to open on Studocu

Tổng hợp bài tập KTMT

Mục lục

<i>Phần 1: Phân tích một đoạn code.....</i>	<i>2</i>
<i>Phần 2: Bài tập bộ nhớ Cache.....</i>	<i>6</i>
<i>Phần 3: Lập trình hợp ngữ.....</i>	<i>8</i>
<i>Phần 4: Giải mã địa chỉ bộ nhớ</i>	<i>36</i>

Phần 1: Phân tích một đoạn code (chế độ địa chỉ, ý nghĩa, xác định giá trị thanh ghi, pipeline)

- Chế độ địa chỉ: tức thì, trực tiếp, gián tiếp qua thanh ghi, gián tiếp qua ô nhớ, địa chỉ chỉ số, địa chỉ tương đối
- Ý nghĩa của đoạn mã: dựa vào nội dung của từng dòng lệnh và mối liên hệ giữa các dòng lệnh
- Xác định giá trị thanh ghi: thực hiện các phép tính toán dựa vào nội dung câu lệnh
- Pipeline: hiểu 5 giai đoạn thực hiện lệnh: Đọc lệnh (IF), giải mã & đọc toán hạng (ID), thực hiện (EX), truy nhập bộ nhớ (MEM), và lưu kết quả (WB)

Bài 1 : Cho đoạn chương trình sau (R1, R2 là các thanh ghi và lệnh quy ước theo dạng LỆNH <ĐÍCH> <GỐC>):

- (1) STORE -100(R2), R1
- (2) LOAD R1, (00FF)
- (3) COMPARE R3, R4
- (4) JUMP-IF-EQUAL Label
- (5) ADD R3, R4
- (6) ADD R2, 2
- (7) Label:

1. Xác định chế độ địa chỉ và ý nghĩa của từng lệnh;
2. Nêu hướng giải quyết xung đột dữ liệu trong pipeline khi thực hiện đoạn chương trình trên biết mỗi lệnh được chia thành 5 giai đoạn.
3. Giả thiết $R3 \neq R4$ và mỗi giai đoạn thực hiện lệnh đều thực hiện trong thời gian là 0.1ns, so sánh thời gian CPU chạy hết 6 lệnh đầu tiên trong trường hợp không sử dụng cơ chế pipeline và có sử dụng cơ chế pipeline trong ý 2.

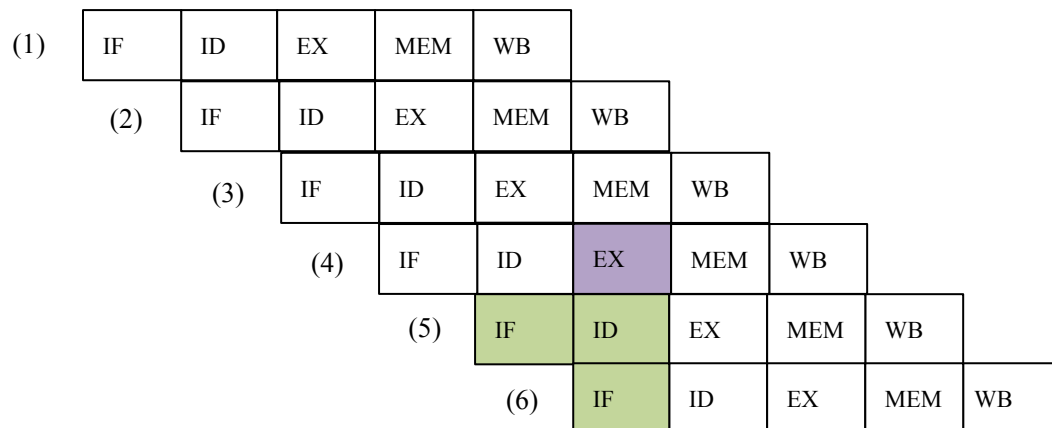
Giải

1. Chế độ địa chỉ và ý nghĩa của từng lệnh
 - (1) STORE -100(R2), R1 :
 - Chế độ địa chỉ chỉ số do toán hạng đích tạo bởi phép cộng giữa 1 hằng số và thanh ghi ($-100(R2) \rightarrow M[R2 - 100]$)
 - Ý nghĩa câu lệnh: Gán trực tiếp giá trị của R1 vào ô nhớ $M[R2-100]$
 - (2) LOAD R1, (00FF) :
 - Chế độ địa chỉ gián tiếp qua ô nhớ do toán hạng nguồn có địa chỉ lưu trong ô nhớ 00FF
 - Ý nghĩa câu lệnh: Gán giá trị trong ô nhớ có địa chỉ lưu ở ô nhớ 00FF vào thanh ghi R1
 - (3) COMPARE R3, R4
 - Chế độ địa chỉ trực tiếp do 2 toán hạng là 2 thanh ghi
 - Ý nghĩa câu lệnh: So sánh R3 và R4.

- (4) JUMP-IF-EQUAL Label
 - Ý nghĩa câu lệnh: Nếu $R3 == R4$ thì nhảy đến nhãn Label
- (5) ADD R3, R4
 - Chế độ địa chỉ trực tiếp do 2 toán hạng là 2 thanh ghi (R3, R4)
 - Ý nghĩa câu lệnh: Gán giá trị thanh $R3 = R3 + R4$
- (6) ADD R2, 2
 - Chế độ địa chỉ trực tiếp do có toán hạng đích là thanh ghi (R2) và toán hạng nguồn là địa chỉ ô nhớ (M[2])
 - Ý nghĩa câu lệnh: Gán giá trị thanh $R2 = R2 + M[2]$

2. Hướng giải quyết xung đột dữ liệu:

- Branch Hazard ở (4) (liên quan đến lệnh nhảy)
- Cách khắc phục: chèn (1) và (2) dưới câu lệnh (4).



3. Thời gian thực hiện 6 câu lệnh khi không sử dụng pipeline

$$5 \cdot n \cdot t = 5 \cdot 6 \cdot 0.1 = 3 \text{ ns}$$

Thời gian thực hiện 6 câu lệnh khi sử dụng pipeline

$$(n+4) \cdot t = (6+4) \cdot 0.1 = 1 \text{ ns}$$

Bài 2: Cho đoạn chương trình hợp ngữ sau

```

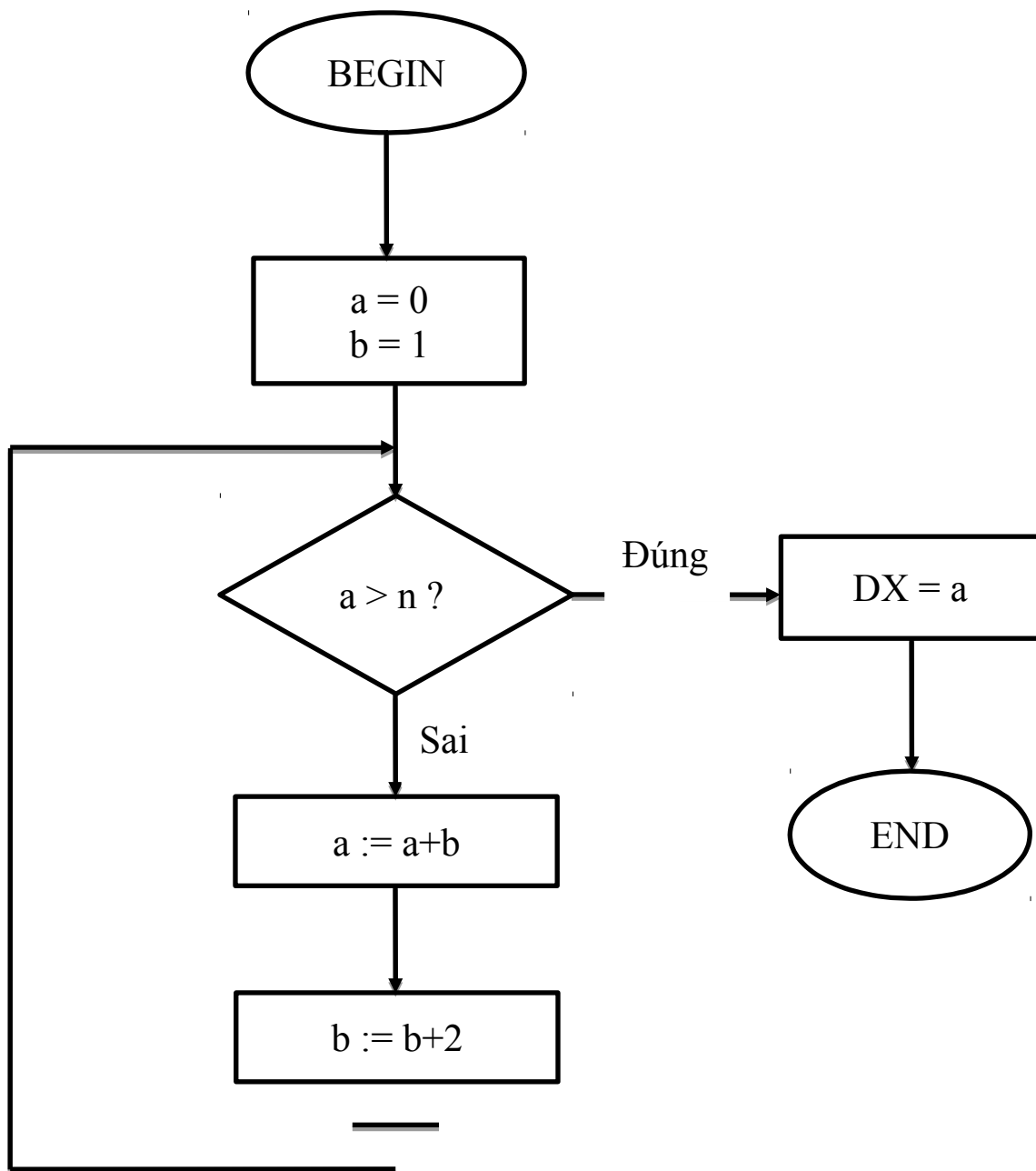
BEGIN:      ADD AX, 0
            ADD BX, 1
LOOP:      CMP AX, CX
            JG FINISH
            ADD AX, BX
            ADD BX, 2
            JMP LOOP
FINISH:     ADD DX, AX
  
```

Giả thiết thanh ghi CX chứa biến số nguyên n và thanh ghi DX để lưu kết quả, thanh ghi AX và BX lưu các biến a, b của chương trình.

1. Vẽ lưu đồ và giải thích ý nghĩa đoạn mã hợp ngữ trên theo các biến a, b, n
2. Tính tổng số lệnh và giá trị thanh ghi DX dưới dạng mã Hexa khi chạy đoạn hợp ngữ trên biết $CX = 0005H$

Giải

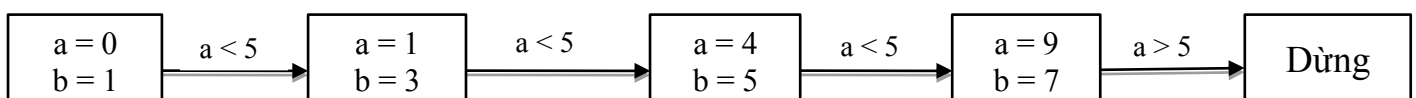
1. Vẽ lưu đồ



Ý nghĩa đoạn mã hợp ngữ: Tính tổng các số nguyên dương lẻ (1,3,5,...) sao cho tổng của chúng nhỏ nhất và tổng phải lớn hơn n. (hay tìm số chính phương nhỏ nhất và lớn hơn n).

2. CX = 0005H = 5₁₀

* Tính số lần lặp



Tổng số lệnh đã thực hiện:

$$2 + 3*5 + 2 + 1 = 20 \text{ (lệnh)}$$

ADD AX, 0	3 vòng	CMP AX, CX	ADD DX, AX
ADD BX, 1	lặp	JG FINISH	

Giá trị thanh ghi DX = a

$$= 9_{10} = 0009H$$

Bài 3: Cho đoạn chương trình sau (R1, R2 là các thanh ghi và lệnh quy ước theo dạng LỆNH <ĐÍCH> <GỐC>):

- (1) LOAD R2, #400
- (2) LOAD R1, #1200
- (3) STORE (R1), R2
- (4) SUBTRACT R2, #20
- (5) ADD 1200, #10
- (6) ADD R2, (R1)

1. Nêu ý nghĩa của từng lệnh;
2. Xác định giá trị của thanh ghi R2 sau khi thực hiện xong lệnh số (6)
3. Nêu một hướng giải quyết xung đột dữ liệu trong pipeline khi thực hiện đoạn chương trình trên biết rằng mỗi lệnh được chia thành 5 giai đoạn trong pipeline: Đọc lệnh (IF), giải mã & đọc toán hạng (ID), thực hiện (EX), truy nhập bộ nhớ (MEM), và lưu kết quả (WB).

Giải

1. Ý nghĩa các câu lệnh

LOAD R2, #400 : gán giá trị thanh R2 = 400

LOAD R1, #1200 : gán giá trị thanh R1 = 1200

STORE (R1), R2 : gán trực tiếp giá trị ô nhớ M[R1] = R2
(M[1200] = 400)

SUBTRACT R2, #20 : gán giá trị thanh R2 := R2 - 20
(R2 = 400 - 20 = 380)

ADD 1200, #10 : cộng 10 vào ô nhớ M[1200] rồi lưu vào M[1200]
(M[1200] = 400 + 10 = 410)

ADD R2, (R1) : cộng giá trị ô nhớ M[1200] vào R2 rồi lưu vào R2
(R2 = 380 + M[1200] = 380 + 410 = 790)

2. Giá trị thanh R2 sau khi thực hiện xong lệnh số (6) R2 = 790

3. Giải quyết xung đột dữ liệu:

- Xung đột dữ liệu giữa (1) và (3) ; giữa (2) và (3) ; giữa (3) và (5) ; giữa (5) và (6).
- Một hướng giải quyết xung đột dữ liệu trong pipeline:
 - + Chèn 3 NO-OP giữa (2) và (3)
 - + Chèn 2 NO-OP giữa (4) và (5)
 - + Chèn 3 NO-OP giữa (5) và (6)

Phần 2: Bài tập bộ nhớ Cache

(by Hữu Sơn)

1. Tính tổng số bit cần thiết cho một bộ nhớ cache tổ chức theo ánh xạ trực tiếp kích thước là 64 KByte dữ liệu và một dòng cache có kích thước là 1 từ nhớ, giả sử bus địa chỉ 32 bit?

Giải:

- + Ta có 1 dòng cache 32bit = 4 byte = 2^2 byte $\Rightarrow$ word = 2 bit
- + Số dòng Cache là $L \Rightarrow$ line = 14bit
- + tag = address_size - line - word = 32-14-2 = 16bit
- + Số lượng bit cần thiết cho một dòng Cache là:
bits/block = data bits + tag bits + valid bit = 32 + 16 + 1 = 49 bit
- + Tổng số bit cần thiết cho bộ nhớ Cache là = $L \times$ bits/block = 14×49 bit = 686 bit = 85.75 Kbytes

2. Tổng số bit sẽ cần là bao nhiêu cho bộ nhớ cache sử dụng tập kết hợp được thiết lập 4 way kích thước là 64 KByte dữ liệu và một dòng cache có kích thước là 1 từ nhớ, giả sử bus địa chỉ 32 bit?

Giải:

- + Ta có 1 dòng cache 32bit = 4 byte = 2^2 byte $\Rightarrow$ word = 2 bit
- + Số dòng Cache là
 $L = 4 \Rightarrow$ line = 12bit
- + tag = address_size - line - word = 32-12-2 = 18bit
- + Số lượng bit cần thiết cho một dòng Cache là: bits/block = data bits + tag bits + valid bit = 32 + 18 + 1 = 51bit
- + Tổng số bit cần thiết cho bộ nhớ Cache là = $L \times$ bits/block $\times$ 51 bit = 4×51 bit = 204 bit = 25.5 Kbytes

way

3. Tính tổng số bit cần thiết cho một bộ nhớ cache tổ chức theo ánh xạ trực tiếp kích thước là 64 KB và một dòng cache có kích thước là 8 từ nhớ, giả sử bus địa chỉ 32 bit?

Giải:

- + Ta có 1 dòng cache 8×32 bit = 32 byte = 2^5 byte $\Rightarrow$ word = 5 bit
- + Số dòng Cache là $L \Rightarrow$ line = 11bit
- + tag = address_size - line - word = 32-11-5 = 16bit
- + Số lượng bit cần thiết cho một dòng Cache là: bits/block = data bits+tag bits+valid bit = $8 \times 32 + 16 + 1 = 273$ bit
- + Tổng số bit cần thiết cho bộ nhớ Cache là = $L \times$ bits/block = 11×273 bit = 3003 bit = 375.375 Kbytes

4. Cho máy có dung lượng bộ nhớ chính: 64KB, dòng cache 8 Bytes, bộ nhớ cache được gồm 32 dòng (lines) được tổ chức ánh xạ trực tiếp.

1. Xác định số bit các thành phần địa chỉ của ô nhớ.

2. Ô nhớ có địa chỉ 0D20H trong bộ nhớ chính được nạp vào dòng (lines) nào của cache

Giải

1. $M_{\text{Memory}} = 64 \text{ KB} = 2^6 * 2^{10} \text{ B} = 2^{16} \text{ B}$

+ Tag + line + word = $\log_2 64 * 2^{10} = \log_2 2^{16} = 16 \text{ bit}$

+ Ta có 1 dòng Cache = $8 \text{ B} = 2^3 \text{ B} \Rightarrow \text{word} = 3 \text{ bit}$

+ Số dòng Cache = $32 \text{ dòng} = 2^5 \Rightarrow \text{line} = 5 \text{ bit}$

+ tag = address_size – line – word = $16 - 5 - 3 = 8 \text{ bit}$

2. Ô nhớ 0D20H

0	D	2	0
0 0 0 0	1 1 0 1	0 0 1 0	0 0 0 0
tag		line	word
13		4	0

$\Rightarrow$ ô nhớ 0D20H nằm ở trang 13, dòng 4, ô 0

Phần 3: Lập trình hợp ngữ

(by Tuấn Đạt)

Một số bài tập tham khảo

I. Lập trình chương trình hợp ngữ Assembly

1. Viết chương trình hợp ngữ in ra lời chào Tiếng Anh và Tiếng Việt.
2. Viết chương trình hợp ngữ Assembly cho phép nhập 1 ký tự và in ra màn hình ký tự đó.
3. Viết chương trình hợp ngữ Assembly cho phép nhập 1 chuỗi ký tự và in ra màn hình chuỗi ký tự đó.
4. Viết chương trình hợp ngữ Assembly cho phép nhập 1 ký tự viết thường và in ra màn hình chữ hoa của ký tự đó.
5. Viết chương trình hợp ngữ Assembly cho phép nhập 1 chuỗi ký tự, in ra màn hình chuỗi ký tự đó theo dạng viết hoa và viết thường.
6. Viết chương trình hợp ngữ Assembly cho phép nhập một chuỗi các ký tự kết thúc bởi “#” và yêu cầu in ra màn hình chuỗi ký tự đó theo thứ tự ngược lại.
7. Viết chương trình hợp ngữ Assembly chuyển một số từ hệ cơ số 10 sang hệ nhị phân.
8. Viết chương trình hợp ngữ Assembly chuyển một số từ hệ cơ số 10 sang hệ cơ số 16 (Hexa).
9. Viết chương trình hợp ngữ Assembly cho phép nhập số nhị phân (8 bit) chứa vào trong thanh ghi BL. Chương trình phải kiểm tra ký tự nhập có hợp lệ hay không (ký tự “0” hoặc ký tự “1”). Việc nhập kết thúc khi nhấn # hoặc đủ 8 bit. Xuất ra số đã nhập dưới dạng hệ thập lục phân (hệ 16).
10. Viết chương trình hợp ngữ Assembly yêu cầu đếm chiều dài của một chuỗi ký tự cho trước.
11. Viết chương trình hợp ngữ Assembly tìm giá trị lớn nhất và nhỏ nhất của một mảng số.
12. Viết chương trình hợp ngữ Assembly cho phép nhập vào một số và in ra màn hình giai thừa của số đó.
13. Viết chương trình hợp ngữ Assembly cho phép nhập vào các số và in ra màn hình tổng của các số đó.
14. Viết chương trình hợp ngữ Assembly cho phép nhập vào 2 số và in ra màn hình UCLN và BCNN của hai số đó.
15. Viết chương trình hợp ngữ Assembly cho phép in ra số lượng các số

chia hết cho 11 và tính tổng của các số đó từ một mảng cho trước.

16. Viết chương trình hợp ngữ Assembly tính tổng 2 số kiểu word.

17. Viết chương trình hợp ngữ cho phép nhập vào một mảng gồm 10 số có hai chữ số. Tính tổng các số chia hết cho 7. In tổng thu được ra màn hình dưới dạng thập phân.

18. Viết chương trình hợp ngữ đếm số lần xuất hiện của chuỗi con “ktmt” trong một chuỗi. In kết quả dưới dạng số thập phân.

19. Viết chương trình hợp ngữ cho hai chuỗi ký tự A và B có độ dài là n và m ($n > m$), chỉ ra xâu B có phải là xâu con của xâu A không?

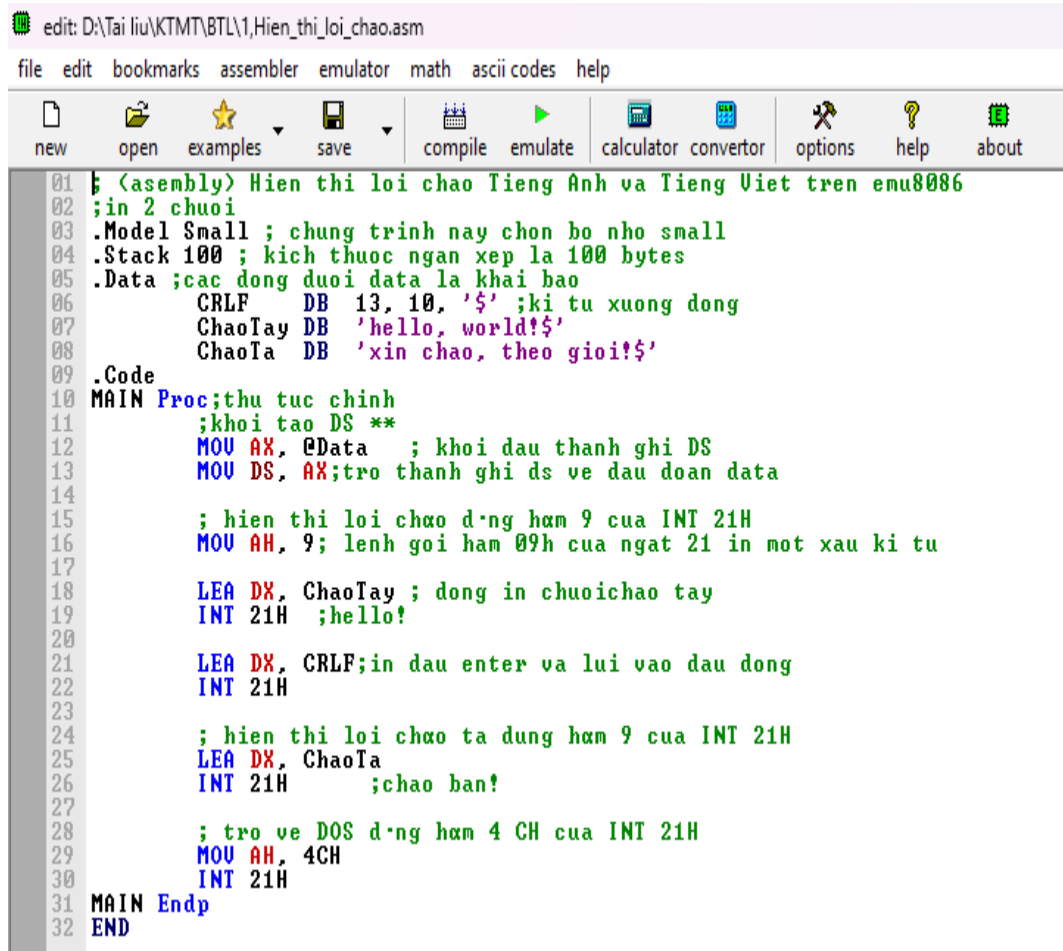
Nếu xâu B là xâu con của xâu A thì chỉ ra vị trí xâu B ở xâu A.

20. Viết chương trình hợp ngữ cho hai chuỗi ký tự A và B có độ dài là n và m ($n > m$), chỉ ra xâu A chứa mấy xâu B.

II. Lập trình điều khiển đèn Led, điều khiển nhiệt kế

I. Lập trình chương trình hợp ngữ Assembly

Câu 1:

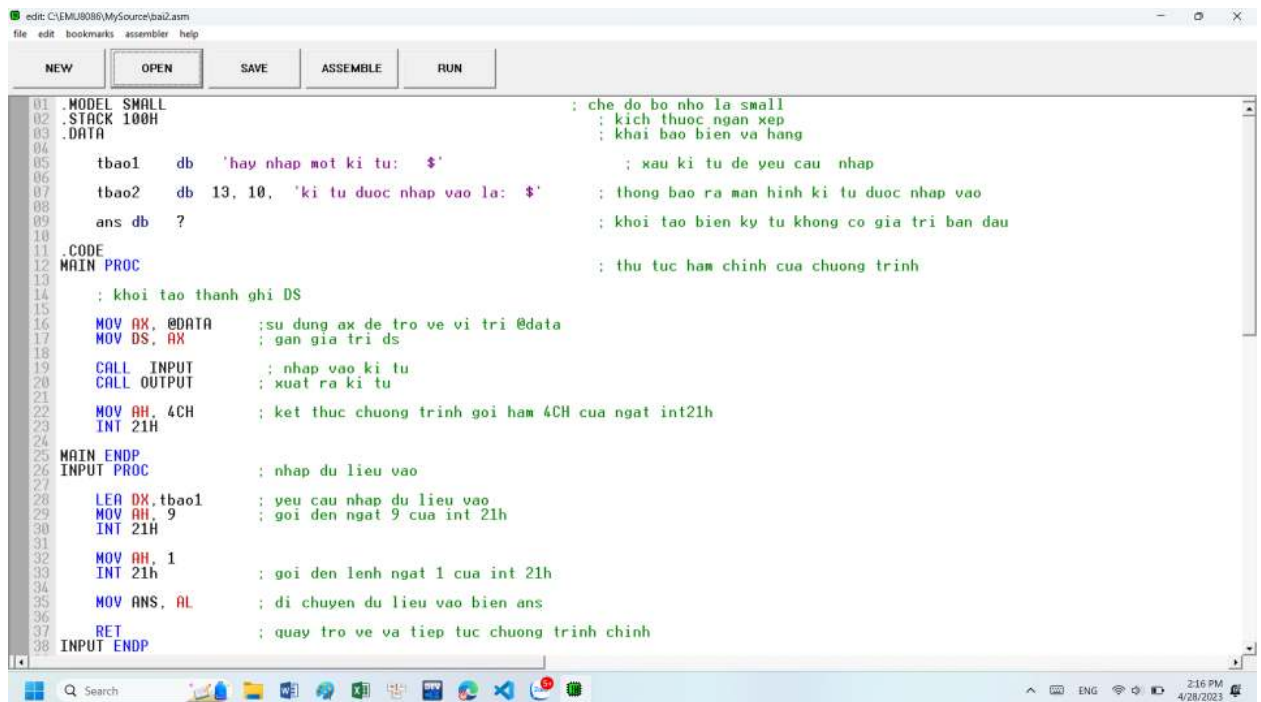


```
edit: D:\Tai lieu\KMTM\BTL\1,Hien_thi_loi_chao.asm
file edit bookmarks assembler emulator math ascii codes help

new open examples save compile emulate calculator convertor options help about

01 ; (assembly) Hien thi loi chao Tieng Anh va Tieng Viet tren emu8086
02 ; in 2 chuoai
03 .Model Small ; chung trinh nay chon bo nho small
04 .Stack 100 ; kich thước ngăn xếp là 100 bytes
05 .Data ; các dòng cuối data là khai báo
06     CRLF DB 13, 10, '$' ; kí tự xuống dòng
07     ChaoTay DB 'hello, world!$'
08     ChaoTa DB 'xin chào, theo giới!$'
09 .Code
10 MAIN Proc; thủ tục chính
11     ; khởi tạo DS **
12     MOV AX, @Data ; khởi đầu thanh ghi DS
13     MOV DS, AX; trở thành ghi ds về đầu đoạn data
14
15     ; hiển thị lời chào dùng hàm 9 của INT 21H
16     MOV AH, 9; lệnh gọi hàm 09h của ngát 21 in một xâu kí tự
17
18     LEA DX, ChaoTay ; dòng in chuỗi chào tay
19     INT 21H ; hello!
20
21     LEA DX, CRLF; in dấu enter và lùi vào đầu dòng
22     INT 21H
23
24     ; hiển thị lời chào ta dùng hàm 9 của INT 21H
25     LEA DX, ChaoTa
26     INT 21H ; chào bạn!
27
28     ; trở về DOS dùng hàm 4 CH của INT 21H
29     MOV AH, 4CH
30     INT 21H
31 MAIN Endp
32 END
```

Câu 2:



```
edit: C:\EMU8086\MySource\bai2.asm
file edit bookmarks assembler help

NEW OPEN SAVE ASSEMBLE RUN

01 .MODEL SMALL ; chế độ bộ nhớ là small
02 .STACK 100H ; kích thước ngăn xếp
03 .DATA ; khai báo biến và hằng
04     tbao1 db 'hãy nhập một kí tự: $' ; xâu kí tự để yêu cầu nhập
05     tbao2 db 13, 10, 'kí tự được nhập vào là: $' ; thông báo ra màn hình kí tự được nhập vào
06     ans db ? ; khởi tạo biến ký tự không có giá trị ban đầu
07
08 .CODE
09 MAIN PROC ; thủ tục hàm chính của chương trình
10     ; khởi tạo thanh ghi DS
11     MOV AX, @DATA ; sử dụng ax để trở về vị trí @data
12     MOV DS, AX ; gán giá trị ds
13
14     CALL INPUT ; nhập vào kí tự
15     CALL OUTPUT ; xuất ra kí tự
16
17     MOV AH, 4CH ; kết thúc chương trình gọi hàm 4CH của ngát int21h
18     INT 21H
19
20 MAIN ENDP
21 INPUT PROC ; nhập dữ liệu vào
22     LEA DX, tbao1 ; yêu cầu nhập dữ liệu vào
23     MOV AH, 9 ; gọi đến ngát 9 của int 21h
24     INT 21H
25
26     MOV AH, 1 ; gọi đến lệnh ngát 1 của int 21h
27     INT 21h
28     MOV ANS, AL ; di chuyển dữ liệu vào biến ans
29
30     RET ; quay trở về và tiếp tục chương trình chính
31 INPUT ENDP
```

```

edit: C:\EMU0806\MySource\ba2.asm
file edit bookmarks assembler help

NEW OPEN SAVE ASSEMBLE RUN

38 INPUT ENDP
39
40 OUTPUT PROC          ; in du lieu ra
41
42     LEA DX, tbao2      ; thông báo in ki tu ra màn hình
43     MOV AH, 9          ; gọi den lenh ngat 9 của int 21h
44     INT 21h
45
46     MOV DL, ANS        ; chuyen du lieu tu bien ans vào thành dl
47
48     MOV AH, 2          ; gọi den lenh ngat 2 của int 21h
49     INT 21h
50
51     RET                ; quay tro ve va tiep tục chương trình chính
52 OUTPUT ENDP
53 END MAIN

```

Câu 3:

```

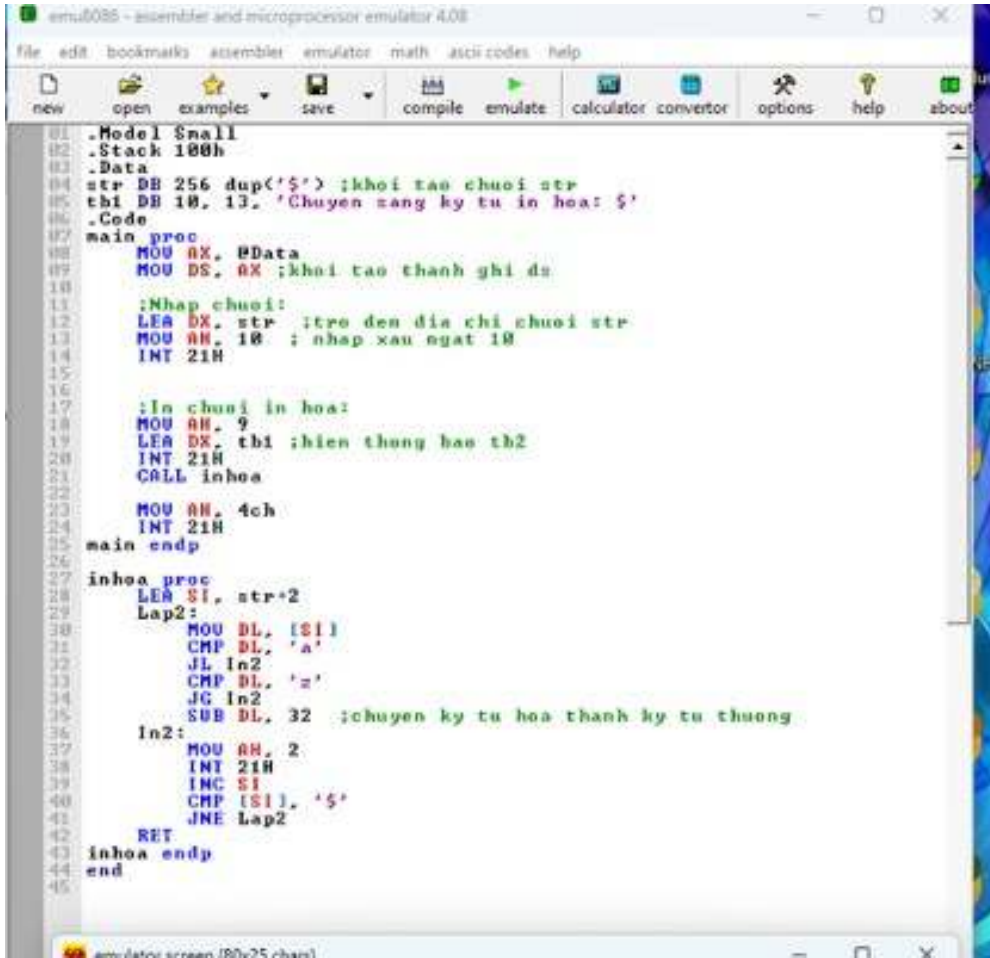
edit: D:\Ki2Nam2\KTM\Emu0806\Project\c3.asm
file edit bookmarks assembler emulator math ascii codes help

new open examples save compile emulate calculator convertor options help about

01 ;Nhap mot chuoì ky tu va in ra màn hình chuoì đó.
02
03 .MODEL SMALL          ;che do của bộ nhớ là small
04 .STACK 100H           ;kích thước ngăn xếp
05 .DATA
06     CRTF DB 13, 10, '$' ;10 là xuống dòng, 13 lùi vào đầu dòng
07     MS1 DB "Nhap vào một chuỗi ký tự: $"
08     MS2 DB 13, 10, "Chuỗi ký tự đã nhập là: $"
09     STR DB 100 DUP('$') ;khởi tạo một mảng ký tự
10 .CODE
11 ;phan chương trình chính
12 MAIN PROC
13     ;khởi tạo thanh ghi DS
14     MOV AX, @data      ;sử dụng AX để trỏ về @data
15     MOV DS, AX         ;DS gán bằng AX
16
17     LEA DX, MS1         ;hiển thị ms1
18     MOV AH, 9           ;sử dụng hàm 9 của ngat INT21H
19     INT 21H
20
21     LEA DX, STR         ;sử dụng hàm 10 của ngat INT 21H
22     MOV AH, 10          ;nhập vào 1 chuỗi ký tự
23     INT 21H
24
25     LEA DX, MS2         ;hiển thị ms2
26     MOV AH, 9
27     INT 21H
28     ;in ra chuỗi ký tự vừa nhập
29     LEA DX, STR + 2     ;đưa DX trỏ về phần tử thứ 2 của mảng
30     MOV AH, 9
31     INT 21H
32
33     MOV AH, 4CH         ;kết thúc chương trình bằng hàm 4CH của ngat INT21H
34     INT 21H
35 MAIN ENDP
36 END MAIN

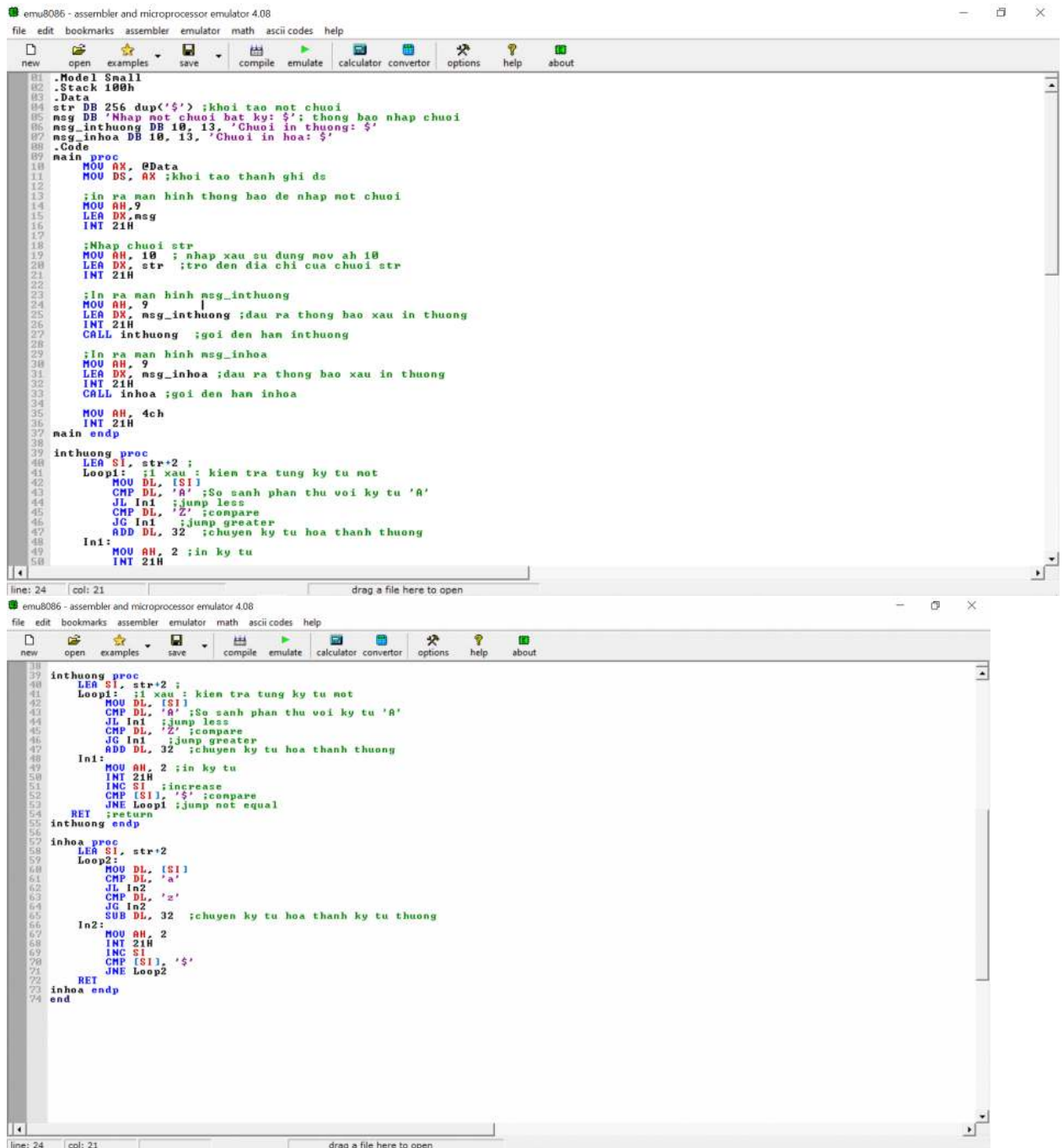
```

Câu 4:



```
01 .Model Small
02 .Stack 100h
03 .Data
04 str DB 256 dup('$') ;khởi tạo chuỗi str
05 tb1 DB 10, 13, 'Chuyen sang ky tu in hoa: $'
06 .Code
07 main proc
08     MOV AX, #Data
09     MOV DS, AX ;khởi tạo thanh ghi ds
10
11     ;Nhập chuỗi:
12     LEA DX, str ;trỏ đến địa chỉ chuỗi str
13     MOV AH, 10 ;nhập xau ngat 10
14     INT 21h
15
16
17     ;In chuỗi in hoa:
18     MOV AH, 9
19     LEA DX, tb1 ;hiển thông báo tb2
20     INT 21h
21     CALL inhoa
22
23     MOV AH, 4ch
24     INT 21h
25 main endp
26
27 inhoa proc
28     LEA SI, str+2
29     Lap2:
30         MOV DL, [SI]
31         CMP DL, 'a'
32         JL In2
33         CMP DL, 'z'
34         JG In2
35         SUB DL, 32 ;chuyen ky tu hoa thanh ky tu thuong
36
37     In2:
38         MOV AH, 2
39         INT 21h
40         INC SI
41         CMP [SI], '$'
42         JNE Lap2
43     RET
44 inhoa endp
45 end
```

Câu 5:



The image shows two windows of the emu8086 - assembler and microprocessor emulator 4.08. The top window displays the main assembly code, and the bottom window displays the code for the `inthuong` and `inhhoa` procedures.

```
.Model Small
.Stack 100h
.Data
str DB 256 dup('$') ;khai tao mot chuoi
msg DB 'Nhap mot chuoi bat ky: $'; thong bao nhap chuoi
msg_inthuong DB 10, 13, 'Chuoi in thuong: $'
msg_inhoa DB 10, 13, 'Chuoi in hoa: $'
.Code
main proc
    MOV AX, @Data
    MOV DS, AX ;khai tao thanh ghi ds

    ;in ra man hinh thong bao de nhap mot chuoi
    MOV AH, 9
    LEA DX, msg
    INT 21h

    ;Nhap chuoi str
    MOV AH, 10 ; nhap xau su dung mov ah 10
    LEA DX, str ;tro den dia chi cua chuoi str
    INT 21h

    ;In ra man hinh msg_inthuong
    MOV AH, 9
    LEA DX, msg_inthuong ;dau ra thong bao xau in thuong
    INT 21h
    CALL inthuong ;goi den ham inthuong

    ;In ra man hinh msg_inhoa
    MOV AH, 9
    LEA DX, msg_inhoa ;dau ra thong bao xau in thuong
    INT 21h
    CALL inhhoa ;goi den ham inhhoa

    MOV AH, 4ch
    INT 21h
main endp

inthuong proc
    LEA SI, str+2 ;
    Loop1: ; xau : kien tra tung ky tu mot
    MOV DL, [SI]
    CMP DL, 'A' ;So sanh phan thu voi ky tu 'A'
    JL In1 ;jump less
    CMP DL, 'Z' ;compare
    JG In1 ;jump greater
    ADD DL, 32 ;chuyen ky tu hoa thanh thuong
    In1:
    MOV AH, 2 ;in ky tu
    INT 21h
    INTH
inthuong endp

inhhoa proc
    LEA SI, str+2
    Loop2:
    MOV DL, [SI]
    CMP DL, 'a'
    JL In2
    CMP DL, 'z'
    JG In2
    SUB DL, 32 ;chuyen ky tu hoa thanh ky tu thuong
    In2:
    MOV AH, 2
    INT 21h
    INC SI
    CMP [SI], '$'
    JNE Loop2
    RET
inhhoa endp
end
```

Câu 6:

```
edit: D:\Tai lieu\KTMT\BTL\6\In_ra_chuoi_ki_tu_nguoc_ket_thuc_bang_dau_#.asm
file edit bookmarks assembler emulator math ascii codes help

new open examples save compile emulate calculator convertor options help about

01 ; <assembly> in chu?i d?o ngu?c khi di?n d?u # tr?n emu8086
02 ;enter la 13 , esc la 27
03 .Model small
04 .Stack 100h
05 .Data
06     str db 50 dup ('$')    ; khoi tao chuoi 50 bytes
07 .code
08 main proc
09     mov ax,@Data
10     mov ds,ax ;khoi tao thanh ghi ds
11     mov cx,0;gan gia tri cho thanh ghi cx=0
12     START:
13         inc cx;increase tang cx len 1
14         mov ah,1 ;nhap 1 ki tu
15         int 21h
16         cmp al,'#';so sanh al voi ki tu # => enter la 13
17         je end ;jump equal :nhay den 'end' neu bang
18         push ax ; them phan tu vao ngan xep
19         jmp START;nhay k dieu kien
20
21     end;ket thuc
22     mov ah,2 ;in ky tu
23     mov dl,0; in dau cach
24     int 21h
25
26     dec cx;decrease:tru cx di 1
27     pop dx;lay phan tu o dinh ngan xep dua vao dx
28     mov ah,2 ;in ky tu
29     int 21h
30
31     cmp cx,1;so sanh cx voi 1
32     jne end;jump not equal :nhay den 'end' neu khong bang
33     ;ve dos
34     mov ah, 4Ch
35     int 21h
36 main endp
37 end
```

Câu 7:

```
edit: C:\EMU8086\MySource\bai7.asm
file edit bookmarks assembler help

NEW OPEN SAVE ASSEMBLE RUN

01 .MODEL SMALL                ; che do bo nho la small
02 .STACK 100H                ; kich thuoc ngan xep
03 .DATA
04     ; khai bao cac bien va hang
05     tb db 10, 13, 'So da nhap dang nhi phan: $'
06     tb1 db 13, 10, 'moi ban nhap so : $'
07     str db 5 dup ('$')      ; nhap vao 1 chuoi toi da 5 ky tu
08 .code
09 MAIN PROC
10
11     MOV AX, @DATA
12     MOV DS, AX              ;khoi tao thanh ghi ds
13
14     MOV AX, '#'
15     PUSH AX                 ;push dau # (ascii) vao trong ngan xep
16
17     lea dx, tb1
18     mov ah, 9
19     int 21h
20
21     ;Nhap so dang chuoi:
22
23     MOV AH, 10
24     LEA DX, str
25     INT 21H
26
27     ;Chuyen chuoi thanh so:
28
29     MOV CL, [str+1]         ; lay so ky tu cua chuoi
30     MOV CH, 0               ; tra ch ve 0
31     LEA SI, str+2           ; tro den dia chi cua ky tu dau tien cua chuoi str
32     MOV AX, 0               ; khoi tao gia tri ban dau bang 0
33     MOV BX, 10              ;bx=10 ;he so nhan
34
35     ;chuyen chuoi thanh so
36
37
38
```



```

edit: C:\MUB086\MySource\bai7.asm
file edit bookmarks assembler help

NEW OPEN SAVE ASSEMBLE RUN

34 MOV BX, 10          ;bx=10 ;he so nhan
35
36
37 ;chuyen chuoi thanh so
38
39
40 thapphan:
41 MUL BX              ;nhan 10
42 MOV DL, [SI]         ;chuyen tu ki tu thanh so
43 SUB DL, '0'          ;ax=ax*dx
44 ADD AX, DX           ;tang si
45 INC SI
46 LOOP thapphan
47
48 MOV CL, 2           ;he so chia
49
50 ;chuyen so thap phan sang nhi phan va day cac so vao ngan xep
51
52 nhiphan:
53 MOV AH, 0           ;phan du =0
54 DIV CL              ;chia ax cho 2
55 PUSH AX             ;day ax vao ngan xep (al*ah)
56 CMP AL, 0           ;so sanh thuong#0 tiep tuc chia
57 JNE nhiphan         ;jump not equal
58
59 MOV AH, 9
60 LER DX, tb;
61 INT 21h
62
63 MOV AH, 2
64
65 output:
66 POP DX              ;lay tung phan tu trong ngan xep
67 CMP DX, 'H'
68 JE Done             ;jump equal
69 MOV DL, DH           ;lay duoc so tu ngan xep
70 ADD DL, '0'
71 INT 21h
72
73 JMP output
74
75 Done:
76 MOV AH, 4Ch
77 INT 21h
78
79 MAIN ENDP
80 END

```


Câu 8:

```

edit: D:\Ki2Nam2\KTMT\Emu8086\Project\C8.asm
file edit bookmarks assembler emulator math ascii codes help
new open examples save compile emulate calculator convertor options help about

001 ;Chuyen tu he DEC sang he HEX
002 .MODEL SMALL
003 .DATA
004
005 CRTF DB 13, 10, '$'
006 MS1 DB "DEC: $"
007 MS2 DB 13, 10, "Convert to HEX: $"
008 str DW 100 dup('$')
009 tmp DW 0 ;bien luu tru tam thoi
010 muoi DW 10 ;luu gia tri 10
011
012 .CODE
013 ;chuong trinh chinh
014 Main Proc
015 ;khởi tạo thanh ghi DS
016 MOV AX, @data ;su dung AX de tro ve @data
017 MOV DS, AX ;gan DS bang AX
018
019 LEA DX, MS1 ;hien thi MS1
020 MOV AH, 9
021 INT 21H
022
023 CALL Input ;nhap vao DEC
024 CALL Convert16 ;chuyen thanh HEX
025
026 ;ket thuc chuong trinh
027 MOV AH, 4CH
028 INT 21H
029
030 Main Endp
031

Input Proc ;nhap vao str va chuyen sang so
;xu li so co nhieu hon 2 chu so
MOV AH, 10 ;nhap vao str
LEA DX, str
INT 21H

LEA SI, str + 2 ;bat dau xau str
MOV CX, [str + 1] ;do dai str nhap vao
XOR CH, CH ;CH = 0

;Loop den khi CX=0->duyet hat xau str
Lap:

MOV AX, tmp ;dua gia tri tu tmp ve AX
MOV BX, [SI] ;ki tu c của si
SUB BX, 48 ;chuyen sang so

MUL muoi ;AX = AX * 10
ADD AX, BX ;AX = AX + BX
MOV tmp, AX ;luu vao tmp
INC SI ;tang index của xau str
LOOP Lap

ret ;return

Input Endp

```

```
060 Convert16 Proc
061     ;chuyen so da nhap tu he DEC sang HEX
062     LEA DX, MS2           ;hien thi MS2
063     MOV AH, 9
064     INT 21H
065
066     MOV AX, tmp           ;AX dang luu tru so can chuyen
067     MOV AH, 0             ;AH = 0
068     MOV BX, 16            ;BX = 16
069     MOV CX, 0             ;CX = 0
070
071     chia:
072         DIV BL             ;lay so hien tai chia cho 16
073         PUSH AX            ;push AX vao stack
074         INC CX             ;tang bien dem CX
075         CMP AL, 0          ;neu thuong AL = 0
076         JE hienthi        ;jump to hienthi
077         MOV AH, 0          ;xoa phan du tai AH
078         JMP chia          ;continue
079
080     hienthi:
081
082         POP AX             ;lay ra tu stack
083         MOV DL, AH         ;truong hop la ki tu tu A->F
084
085
086         CMP DL, 10         ;chuyen den caseA
087         JE caseA
088
089         CMP DL, 11         ;chuyen den caseB
090         JE caseB
091
092         CMP DL, 12         ;chuyen den caseC
093         JE caseC
```

line: 94 col: 10 drag a file here to open

Zalo DEV 10:29 AM 4/30/2023

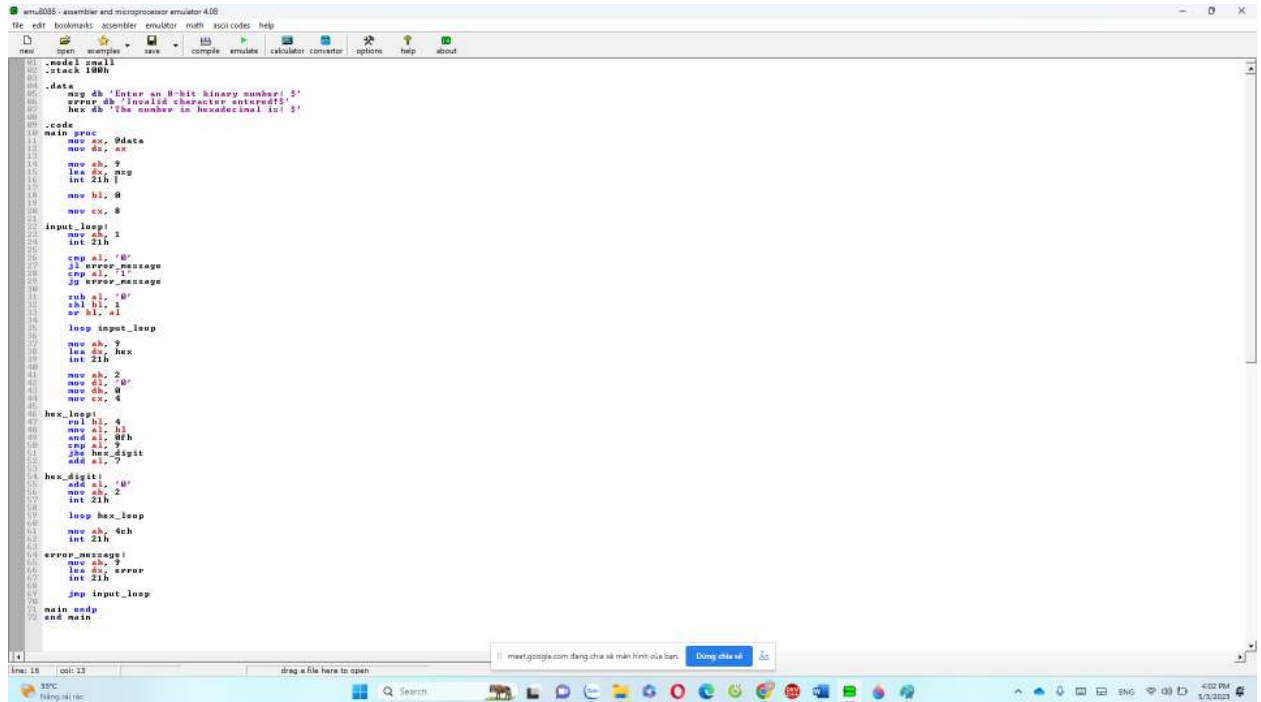
```
095      CMP DL, 13
096      JE caseD      ;chuyen den caseD
097
098      CMP DL, 14
099      JE caseE      ;chuyen den caseE
100
101      CMP DL, 15
102      JE caseF      ;chuyen den caseF
103      ;neu khong phai ki tu tu A->F
104      ADD DL, '0'    ;chuyen so thanh ky tu
105      JMP print
106
107      ;chuyen doi sang ki tu phu hop
108      caseA:
109          MOV DL, 'A'
110          JMP print
111      caseB:
112          MOV DL, 'B'
113          JMP print
114      caseC:
115          MOV DL, 'C'
116          JMP print
117      caseD:
118          MOV DL, 'D'
119          JMP print
120      caseE:
121          MOV DL, 'E'
122          JMP print
123      caseF:
124          MOV DL, 'F'
125          JMP print
126
127      ;in ra ki tu vua xu li
128      print:
129          MOV AH, 2
130          INT 21H
131
132      LOOP hienthi    ;lap den khi da in het
133      ret            ;return
134
135      Convert16 Endp
136      END
137
138
```

ne: 94 col: 10 drag a file here to open 10:31 AM 4/30/2023

```
126
127      ;in ra ki tu vua xu li
128      print:
129          MOV AH, 2
130          INT 21H
131
132      LOOP hienthi    ;lap den khi da in het
133      ret            ;return
134
135      Convert16 Endp
136      END
137
138
```

ne: 123 col: 20 drag a file here to open 10:00 AM 4/27/2023

Câu 9:



```
.model small
.stack 100h

.data
msg db 'Enter an 8-bit binary number: $'
error db 'Invalid character entered!$'
hex db 'The number in hexadecimal is: $'

.code
main proc
mov ax, @data
mov dx, ax
mov ah, 9
lea dx, msg
int 21h
mov hi, 0
mov cx, 8

input_loop:
mov ah, 1
int 21h

cmp al, '0'
jl error_message
cmp al, '1'
jg error_message

sub al, '0'
shl hi, 1
or hi, al

loop input_loop

mov ah, 9
lea dx, hex
int 21h

mov ah, 2
mov al, '0'
mov dx, 0
mov cx, 4

hex_loop:
rol hi, 4
mov al, hi
and al, 0fh
cmp al, 0fh
jbe hex_digit
add al, 7
hex_digit:
add al, '0'

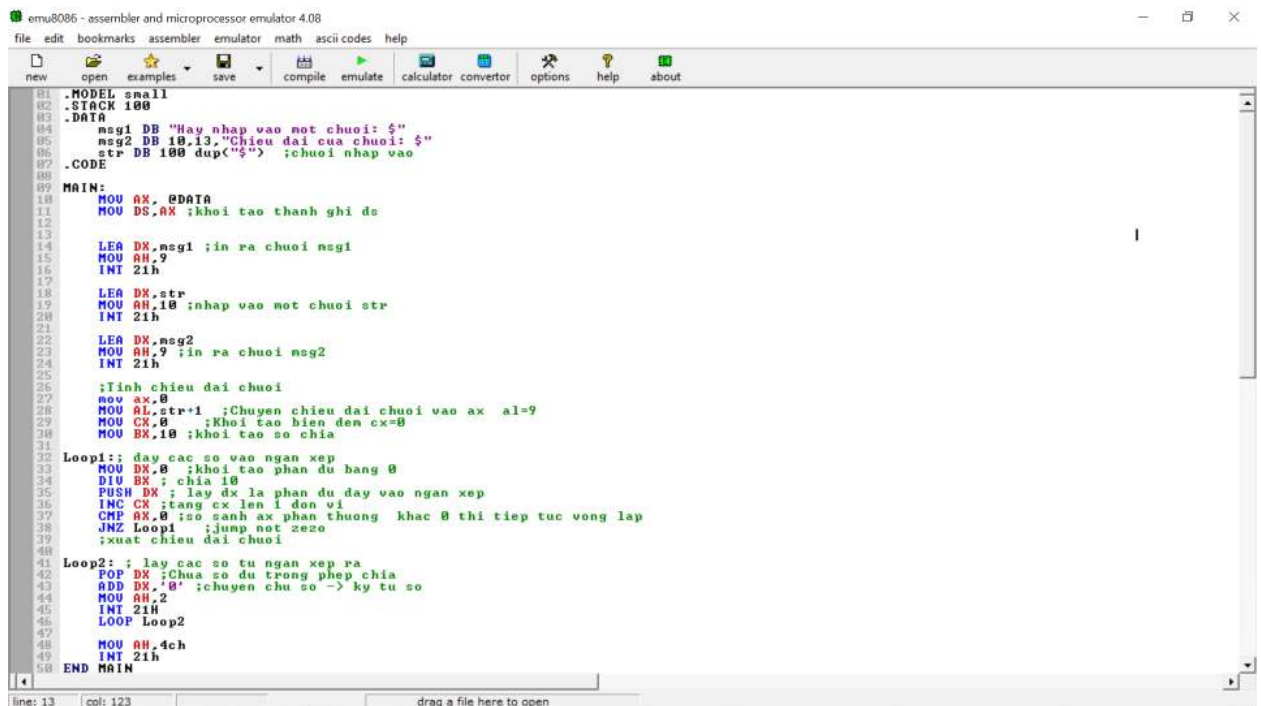
mov ah, 2
mov dx, 0
int 21h
loop hex_loop

mov ah, 4ch
int 21h

error_message:
mov ah, 9
lea dx, error
int 21h
jmp input_loop

main endp
end main
```

Câu 10:



```
.MODEL small
.STACK 100
.DATA
msg1 DB "Hay nhap vao mot chuo: $"
msg2 DB 10,13,"Chieu dai cua chuo: $"
str DB 100 dup("$") ;chuo nhap vao
.CODE
MAIN:
MOV AX, @DATA
MOV DS, AX ;khoi tao thanh ghi ds

LEA DX, msg1 ;in ra chuo msg1
MOV AH, 9
INT 21h

LEA DX, str
MOV AH, 10 ;nhap vao mot chuo str
INT 21h

LEA DX, msg2
MOV AH, 9 ;in ra chuo msg2
INT 21h

;Tinh chieu dai chuo
mov ax, 0
MOV AL, str+1 ;Chuyen chieu dai chuo vao ax al=9
MOV CX, 0 ;Khoi tao bien dem cx=0
MOV BX, 10 ;khoi tao so chia

Loop1: ; day cac so vao ngan xep
MOV DX, 0 ;khoi taophan du bang 0
DIV BX ; chia 10
PUSH DX ; lay dx la phan du day vao ngan xep
INC CX ; tang cx len 1 don vi
CMP AX, 0 ;so sanh ax phan thuong khac 0 thi tiep tục vòng lặp
JNZ Loop1 ;jump not zero
;xuất chieu dai chuo

Loop2: ; lay cac so tu ngan xep ra
POP DX ;Chua so du trong phép chia
ADD DX, 0 ;chuyen chu so -> ky tu so
MOV AH, 2
INT 21h
LOOP Loop2

MOV AH, 4ch
INT 21h
END MAIN
```

Câu 11:

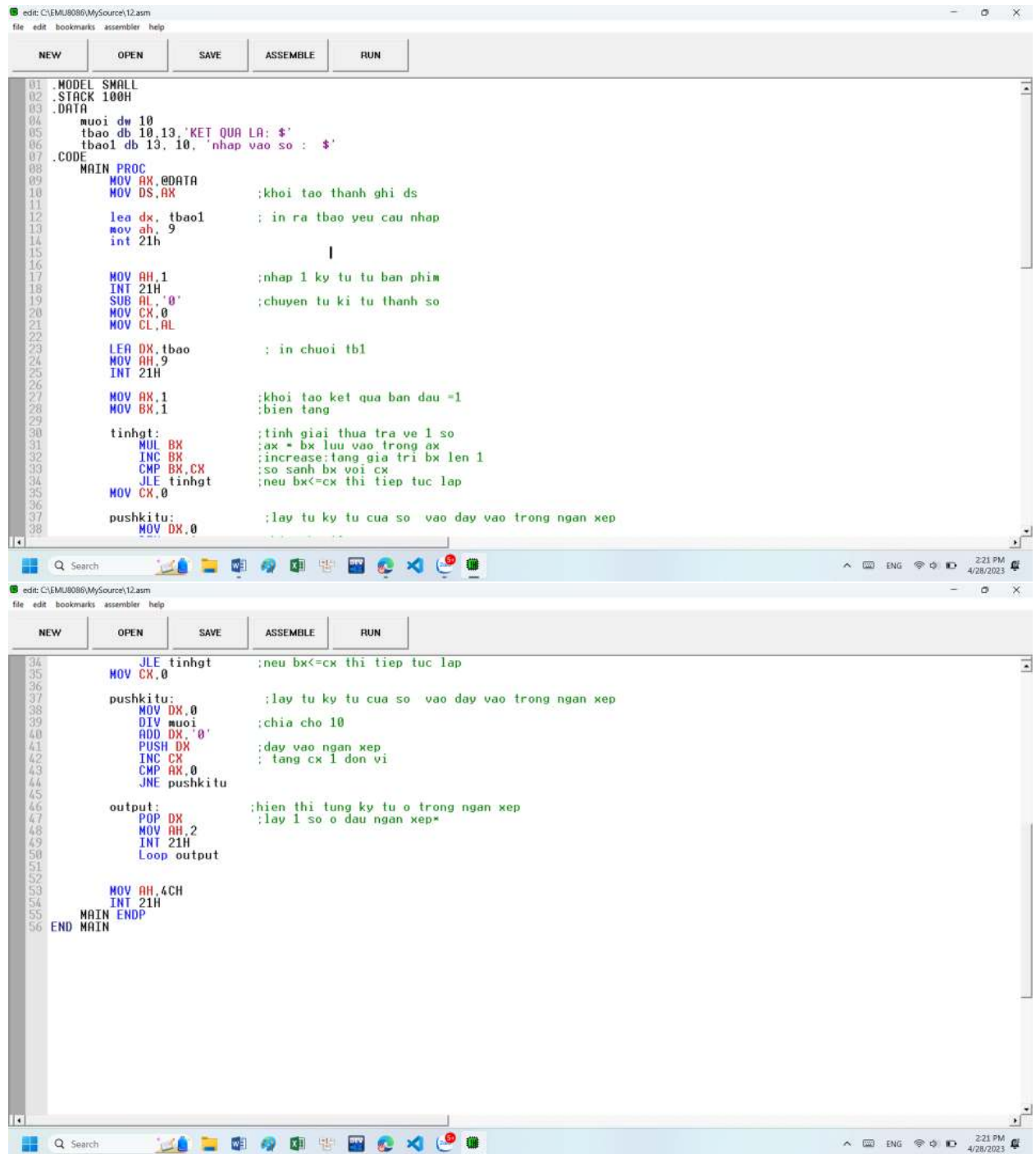
edit: D:\Tai lieu\KTMT\BTL\11,Tim_gia_tri_lon_nhat_va_nho_nhat_cua_mot_day_so.asm

file edit bookmarks assembler emulator math ascii codes help

new open examples save compile emulate calculator convertor options help about

```
01  <assembly> Tim gia tri lon nhat va nho nhat cua mot day so
02  .Model small
03  .Stack 100H
04  .Data
05  list DB 1,2,3,4,5,6,7,8,0
06  .code
07  main proc
08      ; initilize the ds and es registers
09      mov ax, @Data ;
10      mov ds,ax ;
11      mov cx, 9
12  lea si, list      ; dua gia tri dauu tien cua chuoai vao si
13      mov bl, [si]   ; dua dia chi si vao bl
14      inc si         ; tang gia tri si them 1
15  Start:
16      lodsb
17      cmp bl, al ; so sanh al va bl
18      jge BYPASS;  nhay denn BYPASS
19      mov bl, al;   neu al > bl thi gan bl = al;
20  BYPASS:
21      loop Start ; lap
22
23      ; print the max
24      add bl, '0' ; ep kieu so ve kieu ke tu
25      mov dl,bl ; dua gia tri max bl vao dl;
26      mov ah, 2 ; in ra màn hình
27      int 21H
28
29      mov ah, 4CH ; ket thuc chuong trinh
30      int 21H
31  main endp
32  End Main
```

Câu 12:



```
01 .MODEL SMALL
02 .STACK 100H
03 .DATA
04     muoi dw 10
05     tbao db 10,13,'KET QUA LA: $'
06     tbaol db 13, 10, 'nhap vao so : $'
07 .CODE
08     MAIN PROC
09         MOV AX,@DATA
10         MOV DS,AX           ;khởi tạo thanh ghi ds
11
12         lea dx, tbaol       ; in ra tbaol yêu cầu nhập
13         mov ah, 9
14         int 21h
15
16         |
17         MOV AH,1           ;nhập 1 ký tự từ bàn phím
18         INT 21H
19         SUB AL,'0'         ;chuyển từ ký tự thành số
20         MOV CX,0
21         MOV CL,AL
22
23         LEA DX,tbao         ; in chuỗi tb1
24         MOV AH,9
25         INT 21H
26
27         MOV AX,1           ;khởi tạo kết quả ban đầu =1
28         MOV BX,1           ;biến tăng
29
30         tinhgt:            ;tính giai thừa trả về 1 số
31             MUL BX         ;ax = bx lưu vào trong ax
32             INC BX         ;increase:tăng giá trị bx lên 1
33             CMP BX,CX      ;so sánh bx với cx
34             JLE tinhgt     ;neu bx<=cx thì tiếp tục lặp
35             MOV CX,0
36
37         pushkitu:          ;lay tu ký tự của so  vào đây vào trong ngăn xếp
38             MOV DX,0
39
34     JLE tinhgt            ;neu bx<=cx thì tiếp tục lặp
35     MOV CX,0
36
37     pushkitu:            ;lay tu ký tự của so  vào đây vào trong ngăn xếp
38         MOV DX,0
39         DIV muoi          ;chia cho 10
40         ADD DX,'0'
41         PUSH DX           ;day vào ngăn xếp
42         INC CX            ; tăng cx 1 đơn vị
43         CMP AX,0
44         JNE pushkitu
45
46     output:              ;hien thi tung ky tu o trong ngăn xếp
47         POP DX           ;lay 1 số ở đầu ngăn xếp*
48         MOV AH,2
49         INT 21H
50         Loop output
51
52
53     MOV AH,4CH
54     INT 21H
55     MAIN ENDP
56 END MAIN
```


Câu 13:

```

edit: D:\Ki2Nam2\KTM\Emu8086\Project\C13.asm
file edit bookmarks assembler emulator math ascii codes help
new open examples save compile emulate calculator convertor options help about

001 .MODEL SMALL
002 .STACK 100H
003 .DATA
004     sum DW 0                ;dung de luu tong cac so
005     muoi DW 10
006     space DW ' '           ;dau cach
007     str DW 100 dup('$')     ;input cac so > 2 chu so
008     tmp DW 0                ;luu trang thai tam thoi cua cx
009     tmp2 DW 0               ;luu trang thai tam thoi cua ax
010
011     MS1 DW 13, 10, 'Nhap so luong: $'
012     MS2 DW 13, 10, 'Nhap cac so: $'
013     MS3 DW 13, 10, 'Tong cac so vua nhap: $'
014
015 .CODE
016 Main Proc                  ;chuong trinh chinh
017     ;khởi tạo giá trị cho thanh ghi DS
018     MOV AX, @DATA           ;đưa thanh ghi AX trỏ đến @data
019     MOV DS, AX              ;gán DS bằng AX
020
021     LEA DX, MS1              ;hiển thị MS1
022     MOV AH, 9
023     INT 21H
024     ;nhập vào số lượng các số cần tính tổng
025     CALL Input
026     MOV CX, AX               ;CX = số lượng số cần tính tổng
027     XOR CH, CH               ;CH = 0
028     LEA DX, MS2              ;hiển thị MS2
029     MOV AH, 9
030     INT 21H
031     ;nhập vào các số cần tính tổng
032     Lap:
033         MOV tmp, CX           ;lưu trữ tạm thời CX để gọi hàm input
034         CALL Input
035
036         MOV CX, tmp           ;trả lại giá trị cho CX
037         DEC CX                ;CX -= 1
038         MOV BX, sum           ;BX = sum : tổng tính tới hiện tại
039         ADD AX, BX            ;sum = AX + sum
040         MOV sum, AX
041
042         MOV AH, 9             ;hiển thị space cho đẹp
043         LEA DX, space
044         INT 21h
045
046         CMP CX, 0             ;nếu CX = 0 thì dừng
047         JNE lap
048         ;gọi hàm output
049         CALL Output
050
051         MOV AH, 4CH           ;kết thúc chương trình
052         INT 21H
053 Main Endp
054
line: 83 col: 9 drag a file here to open
10:15 AM 4/27/2023
10:16 AM 4/27/2023

```

```

055 Input Proc
056 ;nhap vao str va chuyen sang so
057 MOV AH, 10 ;nhap vao str
058 LEA DX, str
059 INT 21H
060
061 LEA SI, str + 2 ;bat dau xau str
062 MOV CX, [str + 1] ;do dai str nhap vao
063 XOR CH, CH ;CH = 0
064 MOV AX, 0
065 MOV tmp2, AX
066
067 Lap2:
068 MOV BX, 0
069 MOV AX, tmp2 ;dua gia tri tu tmp2 ve AX
070 MOV BX, [SI] ;ki tu c cua str
071 MOV BH, 0
072 SUB BX, '0' ;chuyen sang so
073
074 MUL muoi ;AX = AX * 10
075 ADD AX, BX ;AX = AX + BX
076 MOV tmp2, AX ;luu vao tmp2
077 INC SI ;tang index cua xau str
078 LOOP Lap2
079
080 ret ;return
081 Input Endp

```

line: 83 col: 9

```

083 Output Proc
084
085 LEA DX, MS3 ;hien thi MS3
086 MOV AH, 9
087 INT 21H
088
089 MOV AX, SUM ;AX = sum
090
091 MOV BX, 10 ;BX = 10
092 MOV CX, 0 ;CX = 0 : dem so luong push vao stack
093
094 chia:
095
096 MOV DX, 0
097 DIV BX ;AX/10 phan du luu tai DX
098 PUSH DX ;push DX vao stack
099 INC CX ;CX++
100 CMP AX, 0 ;neu AX = 0 thi jump den hienthi
101 JE hienthi
102 JMP chia ;tiếp tục chia
103
104
105 hienthi:
106 POP DX ;lay DX ra khoi stack
107 ADD DL, 48 ;chuyen tu so sang ki tu
108 MOV AH, 2 ;in ra ki tu vua xu li
109 INT 21H
110 DEC CX ;CX--
111 CMP CX, 0 ;neu CX = 0 thi dung
112 JNE hienthi
113
114 ret
115 Output Endp
116

```

line: 116 col: 3

Câu 14:

```
.model small
.stack 100h

.data
    thi db "Nhap so thu 1: 5"
    th2 db 10,15, "Nhap so thu 2: 5"
    th3 db 10,15, "Nút chung lên nhát là: 5"
    th4 db 10,15, "Nút chung nhát là: 5"
    x dw ?
    y dw ?
    z dw ?
    t dw ?

.code
main proc
    mov ax,@data
    mov dx,ax
    ;nhap so thu 1
    mov bh,7
    lea dx,th1
    int 21h
    call nhap_so
    mov ax,x
    ;nhap so thu 2
    mov bh,7
    lea dx,th2
    int 21h
    call nhap_so
    mov ax,x
    mov t,ax
    ; nút chung lên nhát
    mov bh,7
    lea dx,th3
    int 21h
    mov ax,z
    mov t,ax
    while:
        cmp ax,bx
        je ucim
        jg ucim
        jmp while
    lea dx,bx
    jmp while
ucim:
    mov x,ax
    call print
    ; nút chung nhát
    mov bh,7
    lea dx,th4
    int 21h
    mov ax,x
    mov k,ax
    mov t,ax
    mul bx,k
    div bx
    mov x,ax
    call print
    mov ah,4
    int 21h
main endp
```

```

mov ax,x
mov k,ax
mov ax,z
mov bx,t
mul bx,k
div bx
mov x,ax
call print
mov ah,4
int 21h
main endp

;nhap so
nhap_so proc
    mov x,0
    mov y,0
    mov bx,10
    nhap:
        mov ah,1
        int 21h
        cmp al,13
        je chuat
        sub al,30h
        mov y,ax
        mov ax,x
        mul bx
        add ax,y
        mov x,ax
        jmp nhap
    thuat:
        ret
nhap_so endp

;hien thi
hien_thi proc
    mov ax,x
    mov bx,10
    mov cx,0
    chia:
        mov dx,0
        div bx
        push dx
        inc cx
        cmp ax,0
        je hienthi
        jmp chia
    hienthi:
        mov dx,30h
        mov ah,2
        int 21h
        dec cx
        cmp cx,0
        je xong
        jmp hienthi
    xong:
        ret
hien_thi endp
end main
```

```

file edit bookmarks assembler emulator math ascii codes help
new open examples save compile emulate calculator convertor options help about

.model small
.stack 100h
.data
    tb1 db "nhap so thu 1: %"
    tb2 db 10,13, "nhap so thu 2: %"
    tb3 db 10,13, "so chung nho nhut la: %"
    tb4 db 10,13, "hoi chung nho nhut la: %"
    x dw ?
    y dw ?
    t dw ?
    h dw ?
.code
main proc
    mov ax,@data
    mov ds,ax

    ;nhap so thu 1
    mov ah,9
    lea dx,tb1
    int 21h

    call nhap_so
    mov ax,x
    mov dx,ax
    ;nhap so thu 2
    mov ah,9
    lea dx,tb2
    int 21h

    call nhap_so
    mov ax,x
    mov t,ax

    ;so chung nho nhut
    mov ah,9
    lea dx,tb3
    int 21h

    mov ax,t
    mov hx,t

    while:
        cmp ax,hx
        je ucini
        jl less
        sub ax,hx
        jmp while

    less:
        sub hx,ax
        jmp while

    ucini:
        mov x,ax
        call print

    ; hoi chung nho nhut
    mov ah,9

```

Câu 15:

```

emu8086 - assembler and microprocessor emulator 4.08
file edit bookmarks assembler emulator math ascii codes help
new open examples save compile emulate calculator convertor options help about

.model small
.stack 100h
.data
    mang1 db 7,11,66,233,77,43,55,123,121
    msg1 db "Tong cac so chia het cho 11 la: %"
    msg2 db 10,13, "So luong cac so chia het cho 11 la: %"
    x dw ?
    y dw ?
.code
main proc
    mov ax,@data
    mov ds,ax ;khai tao thanh ghi ds

    lea dx,msg1 ;lea chuoai msg vao thanh ghi dx
    mov ah,9 ;su dung han nove ah,9 de in ra chuoai msg1
    int 21h

    ;in ra tong cua cac so chia het cho 11
    mov x,0 ;khai tao bien x = 0
    mov cx,9 ;khai tao bien den cx bang voi so phan tu trong mang
    lea si,mang1 ;su dung thanh ghi si de tro toi dia chi cua mang
    mov bx,11 ;khai tao so chia

    while:
        mov dx,0 ;khai tao thanh ghi dx = 0
        mov ax,[si] ;khai tao thanh ghi ax voi dia chi si
        xor ah,ah ;xoa thanh ghi ah
        div bx ;chia cho bien bx da khai tao ben tren la bang 11
        cmp dx,0 ;so sanh neu chia het thi phan du la 0
        je tang
        jmp tieptuc

    tang:
        mov ax,[si]
        xor ah,ah
        add x,ax ;cong x voi phan tu trong mang duoc luu o ax
        inc y ;tang bien y len 1 don vi
        tieptuc:
        inc si ;tang bien si len 1 don vi
        loop while

    mov bx,10 ;khai tao bx = 10 de chia lay so du
    mov ax,x ;khai tao ax = x
    mov cx,0
    dem1:
        mov dx,0 ;khai tao phan du = 0
        div bx ;chia 10 lay du;

```

```

emu8086 - assembler and microprocessor emulator 4.08
file edit bookmarks assembler emulator math ascii codes help
new open examples save compile emulate calculator convertor options help about

41      tieptuc:
42          inc si ;tang bien si len 1 don vi
43          loop while
44
45      mov bx,10; khoi tao bx = 10 de chia lay so du
46      mov ax,x; khoi tao ax = x
47      mov cx,0;
48      den1:
49          mov dx,0; khoi taophan du = 0
50          div bx ;chia 10 lay du;
51          push dx; lay dx la phan tu day vao ngan xep
52          inc cx; tang cx len 1 don vi
53          cmp ax,0; so sanh ax phan thuong khac 0 thi tiep tục
54          jnz den1
55      den2:
56          pop dx; chua so du top trong phep chia
57          add dx,'0'; cong dx voi ky tu '0' de chuyen so thanh mot ky tu
58          mov ah,2; su dung ham ah2 de in ra 1 ky tu
59          int 21h
60          loop den2
61
62      ;in ra tong so luong cac so chia het cho 11;
63      lea dx,msg2;
64      mov ah,9
65      int 21h
66
67      mov bx,10; khoi tao bx = 10 de chia lay so du
68      mov ax,y; khoi tao ax = y
69      mov cx,0;
70
71      check1:
72          mov dx,0; khoi tao phan du = 0
73          div bx ;chia 10 lay du;
74          push dx; lay dx la phan tu day vao ngan xep
75          inc cx; tang cx len 1 don vi
76          cmp ax,0; so sanh ax phan thuong khac 0 thi tiep tục
77          jnz check1
78      check2:
79          pop dx; chua so du top trong phep chia
80          add dx,'0'; cong dx voi ky tu '0' de chuyen so thanh mot ky tu
81          mov ah,2; su dung ham ah2 de in ra 1 ky tu
82          int 21h
83          loop check2
84
85      mov ah,4
86      int 21h
87
88      main endp
89
90
line: 89 col: 30 drag a file here to open

```

Câu 16:

```

10 .code
11 main proc
12 mov ax,@data
13 mov ds,ax
14 ;in thong bao nhap so thu nhat
15 Page 34
16 lea dx,tb1
17 mov ah,9
18 int 21h
19 nhap1:
20 mov ah,1
21 int 21h
22 cmp al,13 ;so sanh ky tu vua nhap voi 13
23 je nhap2 ;neu bang nhap so thu 2
24 sub al,30h ;doi ky tu sang so
25 mov ah,0 ;xoa bit cao
26 mov cx,ax ;cat so vua nhap vao cx
27 mov ax,so1 ;dua bien sf 1 vao kiou byte de chuan bi nhann vao 10
28 mov bx,10 ;gan bx =10
29 mul bx ;nhin ax voi 10
30 add ax,cx ;cng ket qua vua nhan voi so vua nhap ket qua c't vao ax
31 mov so1,ax ; cat ket qua vao bien sf1
32 jmp nhap1
33 nhap2:
34 lea dx,tb2 ;hin thng bpo nh'p sf thu 2
35 mov ah,9
36 int 21h
37 nhap: mov ah,1 ;nhap sf thu 2
38 int 21h
39 cmp al,13 ;so s'nh k2 tu vua nh'p voi 13
40 je tinhtong ;neu bang th' t'nh t'ng
41 sub al,30h ;chuy'nn k2 tu sang dang sf
42 mov ah,0 ;xob b'it cao
43 mov cx,ax ;c't ket qua vua nhap vao cx
44 mov ax,so2 ;dua bien sf 2 vao kiou byte
45 mov bx,10 ;gan bx=10
46 mul bx ;nhin ket qua vua nhap voi 10
47 add ax,cx ;cng ket qua vua nh'nn vao sf vua nh'p
48 mov so2,ax ;c't ket qua vao bien sf 2
49 jmp nhap
50 tinhtong:
51 mov dx,tong
52 mov ax,so1 ;dua bien sf 1 ra thanh ghi ax
53 mov bx,so2 ;dua bien sf 2 ra thanh ghi bx
54 add ax,bx ;cng ax voi bx ket qua cat vao ax
55 mov tong,ax ;dua ket qua tu ax vao bien t'ng
56 inso: mov ah,9 ;hin thng bpo in t'ng
57 lea dx,tb3
58 int 21h
59 mov ax,tong ;dua ket qua trongv bien t'ng ra thanh ghi ax
60 mov dx,0 ;xoa bit cao dx
61 mov bx,10 ;gan bx=10
62 mov cx,0 ;kh'oi tao bien d'm
63 chia: div bx ;l'y ket qua chia cho 10
64 push dx ;du o dx d'y vao ngan x'p
65 inc cx ;tang bien d'm
66 cmp ax,0 ;so sanh thuong voi 0
67 je hienkq ;neu bang th' hin ket qua
68 xor dx,dx ;xoa bit cao trong dx
69 jmp chia
70 hienkq: pop dx ;l'y du trong ngan x'p ra khoi dx
71 add dl,30h ;chuy'nn sf thanh dang k2 tu
72 mov ah,2 ;in t'ng
73 int 21h
74 loop hienkq
75 ra: mov ah,4ch
76 int 21h
77 Main endp
78 End mai

```

Câu 17:

```

edit: C:\MUB080\MySource\17.asm
file edit bookmarks assembler help

NEW OPEN SAVE ASSEMBLE RUN

001 |MODEL Tiny
002 |STACK 100h
003 |.DATA
004 |    sum dw 0 ;luu tong cac so
005 |    muoi dw 10
006 |    bay dw 7
007 |    space dw ' '
008 |    str dw 100 dup('$') ;LUU CAC SO DUOI DANG XAU KI TU
009 |    tmp dw 0 ;CAC BIEN NHO TAM
010 |    tmp2 dw 0
011 |    tmp3 dw 0
012 |    TB2 dw 13, 10, 'Nhap 10 so : $'
013 |    TB3 dw 13, 10, 'Tong cac so vua nhap: $'
014 |
015 |.CODE
016 |MAIN PROC ;chuong trinh chinh
017 |
018 |    MOV AX, @DATA
019 |    MOV DS, AX
020 |
021 |
022 |
023 |    MOV CX, 10
024 |    MOV CH, 0
025 |
026 |    LEA DX, TB2 ;HIEN THI THONG BAO MAN HINH YEU CAU NHAP 10 SO
027 |    MOV AH, 9
028 |    INT 21h
029 |
030 |    Lap: ;NHAP VAO 10 SO
031 |
032 |    MOV TMP, CX ;LUU LAI CX DE GOI DEN HAM INPUT
033 |    CALL INPUT
034 |
035 |    MOV CX, TMP ;TRA LAI GIA TRI CX
036 |    DEC CX ;CX = CX - 1
037 |    MOV BX, SUM ;GAN GIA TRI TONG HIEN TAI CHO THANH GHI BX
038 |
039 |    ;KIEM TRA CAC SO XEM CO CHIA HET CHO 7 KHONG
040 |
041 |    MOV TMP3, AX ;BO TAM THOI GIA TRI CUA THANH AX VAO BIEN TMP3
042 |    DIV bay ;THUC HIEN PHEP CHIA CHO 7
043 |    CMP DX, 0 ;KIEM TRA XEM CO CHIA HET CHO 7 KHONG
044 |    JNE CONTINUE
045 |
046 |    MOV AX, TMP3 ;TRA LAI TMP3 CHO AX
047 |    ADD AX, BX ;AX = AX + BX
048 |    MOV sum, AX ;SUM = AX
049 |
050 |
051 |    CONTINUE:
052 |
053 |    MOV AH, 9 ;CACH RA 1 KHOANG TRONG
054 |    LEA DX, space
055 |    INT 21h
056 |
057 |
058 |
059 |    CMP CX, 0 ;KIEM TRA XEM CX = 0 ?
060 |    JNE lap ;GOI DEN OUTPUT
061 |    CALL output
062 |
063 |    MOV AH, 4Ch ;KET THUC CT
064 |    INT 21h
065 |MAIN END

```

```

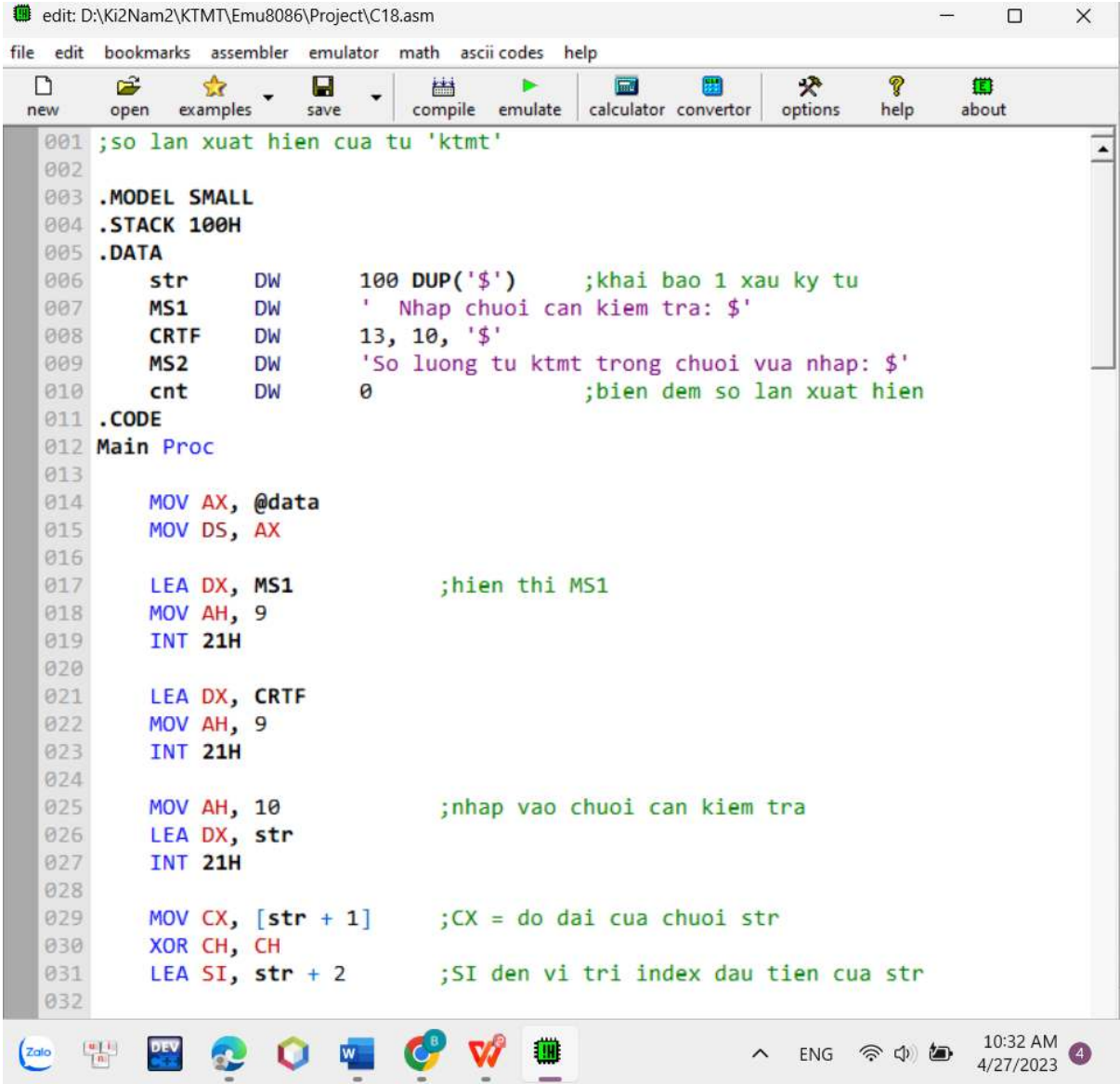
edit: C:\MUB080\MySource\17.asm
file edit bookmarks assembler help

NEW OPEN SAVE ASSEMBLE RUN

064      INT 21H
065  MAIN ENDP
066
067  INPUT PROC                                ;NHAP VAO SO DUOI DANG KI TU VA CHUYEN SANG SO
068
069
070      MOV AH, 10                            ;NHAP VAP XAU STR
071      LEA DX, str
072      INT 21H
073
074      LEA SI, str + 2                        ;BAT DAU CUA CHUOI KI TU
075      MOV CX, [str + 1]                    ;DO DAI STR
076      MOV CH, 0                            ;CH = 0
077      MOV AX, 0
078      MOV tmp2, AX
079
080  Lap2:
081      MOV BX, 0
082      MOV AX, tmp2                          ;DUA GIA TRI TMP2 VE AX
083      MOV BX, [SI]                          ;KI TU THU SI TRONG XAU
084      MOV BH, 0
085      SUB BX, '0'                           ;CHUYEN SANG SO
086
087      MUL muoi                               ;AX = AX * 10
088      ADD AX, BX                            ;AX = AX + BX
089      MOV tmp2, AX                          ;LUU VAO TMP2
090      INC SI                                ;SI = SI + 1
091      LOOP Lap2
092
093
094      RET
095
096  INPUT ENDP
097
098  OUTPUT PROC
099
100      LEA DX, TB3                           ;HIEN THI THONG BAO 3
101
102
103
104
105
106
107
108
109
110
111
112
113
114
115
116
117
118
119
120
121
122
123
124
125
126
127
128
129
130  end main
131

```


Câu 18:



The screenshot shows an x86-64 assembler editor window titled "edit: D:\Ki2Nam2\KTMT\Emu8086\Project\C18.asm". The window has a menu bar with "file", "edit", "bookmarks", "assembler", "emulator", "math", "ascii codes", and "help". Below the menu bar is a toolbar with icons for "new", "open", "examples", "save", "compile", "emulate", "calculator", "convertor", "options", "help", and "about". The main text area contains assembly code with line numbers from 001 to 032. The code defines a data segment with a string "Nhap chuoai can kiem tra: \$" and its length 13. It then defines a code segment with a "Main Proc" that prints the string and its length using DOS interrupts. The taskbar at the bottom shows various application icons and the system clock indicating 10:32 AM on 4/27/2023.

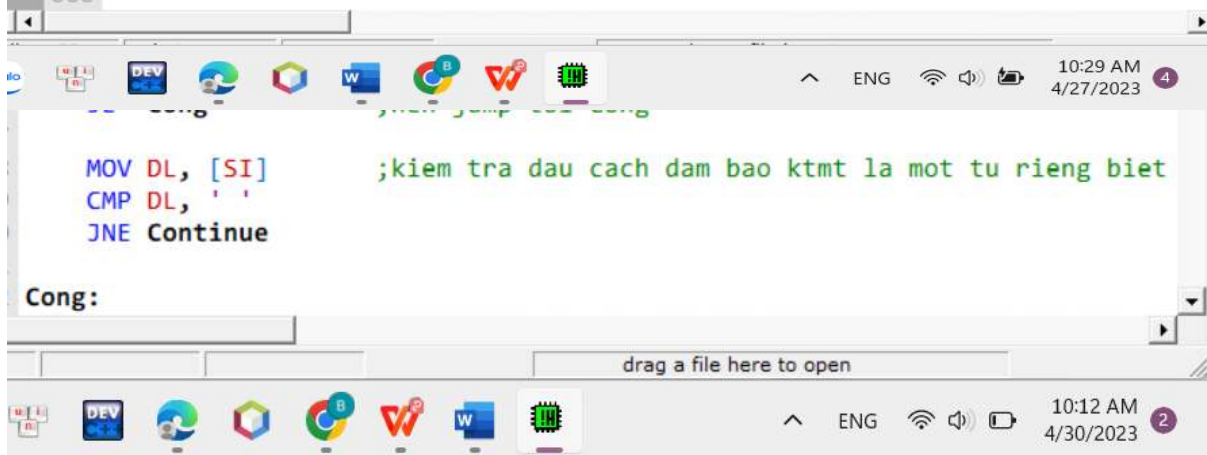
```
001 ;so lan xuất hiện của từ 'ktmt'
002
003 .MODEL SMALL
004 .STACK 100H
005 .DATA
006     str      DW      100 DUP('$')      ;khai báo 1 xâu ký tự
007     MS1      DW      ' Nhap chuoai can kiem tra: $'
008     CRTF     DW      13, 10, '$'
009     MS2      DW      'So luong tu ktmt trong chuoai vua nhap: $'
010     cnt      DW      0                  ;bien dem so lan xuất hiện
011 .CODE
012 Main Proc
013
014     MOV AX, @data
015     MOV DS, AX
016
017     LEA DX, MS1                        ;hiển thị MS1
018     MOV AH, 9
019     INT 21H
020
021     LEA DX, CRTF
022     MOV AH, 9
023     INT 21H
024
025     MOV AH, 10                        ;nhập vào chuỗi cần kiểm tra
026     LEA DX, str
027     INT 21H
028
029     MOV CX, [str + 1]                 ;CX = độ dài của chuỗi str
030     XOR CH, CH
031     LEA SI, str + 2                   ;SI đến vị trí index đầu tiên của str
032
```



```

031
032 | LAP:
033     ;kiem tra lan luot cac ki tu 'k','t','m','t' dung canh nhau
034
035     MOV DL, [SI]      ;so sanh voi ki tu 'k'
036     CMP DL, 'k'
037     JNE Continue     ;neu khong phai jump toi Continue
038     INC SI           ;SI++
039     DEC CX           ;CX--
040     CMP CX, 0        ;neu CX = 0 thi jump den Stop
041     JE Stop
042     ;so sanh voi ki tu 't'
043     MOV DL, [SI]
044     CMP DL, 't'
045     JNE Continue
046     INC SI
047     DEC CX
048     CMP CX, 0
049     JE Stop
050     ;so sanh voi ki tu 'm'
051     MOV DL, [SI]
052     CMP DL, 'm'
053     JNE Continue
054     INC SI
055     DEC CX
056     CMP CX, 0
057     JE Stop
058     ;so sanh voi ki tu 't'
059     MOV DL, [SI]
060     CMP DL, 't'
061     JNE Continue
062     INC SI
063     DEC CX
064     CMP CX, 0        ;neu CX = 0 thi ktmt la chuoi cuoi cua day
065     JE Cong          ;nen jump toi Cong
066

```



```

070
071 Cong:
072     MOV AX, 1           ;Cnt ++
073     ADD Cnt, AX
074     CMP CX, 0
075     JE Stop
076
077 Continue:                ;tiếp tục duyệt lại từ kí tự k
078     INC SI              ;SI ++
079     DEC CX              ;CX--
080     CMP CX, 0
081     JNE LAP             ;jump to Lap
082 Stop:
083
084     CALL Output          ;jump to Output
085
086     MOV AH, 4CH          ;kết thúc chương trình
087     INT 21H
088
089 Main Endp
090

```



Câu 19:

```
emu0386 - assembler and microprocessor emulator 4.06
file edit bookmarks assembler emulator math ascii codes help

new open assemble save compile emulate calculator converter options help about

0001 .model small
0002 .stack 100h
0003
0004 .data
0005     B db 100 dup('$')
0006     n du 2
0007     i du 2
0008     j du 2
0009     found db ?
0010
0011 msg_found db 'Xau B la xau con cua xau A'. '$'
0012 not_found_msg db 'Xau B khong phai la con cua xau A'. '$'
0013 result_msg db 00h, 'Oi tri cua xau B trong xau A la! $'
0014
0015
0016 .code
0017 mov ax, 0data
0018 mov dx, ax
0019
0020 ; Nhap xau A
0021 mov sb, 00h
0022 lea dx, 0
0023 int 21h
0024 ; Nhap xau B
0025 mov sb, 00h
0026 lea dx, 0
0027 int 21h
0028
0029 ; Tinh do dai cua xau A va B
0030 lea si, 0
0031 mov cx, 0
0032 strlen_A:
0033     cmp byte ptr [si], '$'
0034     je strlen_A_done
0035     inc si
0036     jmp strlen_A
0037
0038 strlen_A_done:
0039     mov n, cx
0040
0041 lea si, 0
0042 mov cx, 0
0043 strlen_B:
0044     cmp byte ptr [si], '$'
0045     je strlen_B_done
0046     inc si
0047     jmp strlen_B
0048
0049 strlen_B_done:
0050     mov n, cx
0051
0052 ; Kiểm tra xau B có phải là con của xau A không
0053 lea di, 0
0054 mov found, 0
0055 mov cx, n
0056 mov si, 0
0057 cmp cx, 0
0058 jz not_found
0059
0060 outer_loop:
0061     mov cx, 0
0062     mov i, cx
0063     add i, 1
0064
0065     inner_loop:
0066         cmp si, 0
0067         je found_it
0068         mov si, byte ptr [i + di]
0069         mov di, byte ptr [B + di]
0070         cmp si, di
0071         jne next_outer
0072         inc si
0073         jmp inner_loop
0074
0075 found_it:
0076     mov found, 1
0077     mov pos, i
0078     jmp end
0079
0080 next_outer:
0081     add di, 1
0082     dec cx
0083     cmp cx, 0
0084     jpe outer_loop
0085
0086 not_found:
0087     mov found, 0
0088
0089 end:
0090 ; in kết quả?
0091 mov sb, 00h
0092 cmp found, 1
0093 je found_msg
0094 lea dx, not_found_msg
0095 jmp print_msg
0096
0097 found_msg:
0098     lea dx, found_msg
0099     jmp print_msg
0100
0101 print_msg:
0102     int 21h
0103
0104 mov ch, 00h
0105 cmp found, 1
0106 jne end_program
0107 mov ax, pos
0108 lea dx, result_msg
0109 jmp print_result
0110
0111 print_result:
0112     int 21h
0113
0114 end_program:
0115 mov sb, 4Ch
0116 int 21h
0117
0118 end
```

```

new open examples save compile emulate calculator converter options help about
0001 .model small
0002 .stack 100h
0003
0004 .data
0005     A db 100 dup('5')
0006     B db 100 dup('5')
0007     n du 7
0008     m du 7
0009     i du 7
0010     j du 7
0011     found db 7
0012     msg_found db 'Xau B la xau con cua xau A', '5'
0013     msg_notfound db 'Xau B khong phai la xau con cua xau A', '5'
0014     result msg db 00h, '5' tri cua xau B trong xau A la: 5'
0015
0016 .code
0017 mov ax, 0data
0018 mov es, ax
0019
0020 ;Nhap xau A
0021 mov ah, 0ah
0022 int 21h
0023
0024 ;Nhap xau B
0025 mov ah, 0ah
0026 int 21h
0027
0028 ;Tinh do dai cua xau A va B
0029 lea si, A
0030 mov cx, 0
0031 strlen_A:
0032     cmp byte ptr [si], '5'
0033     je strlen_A_done
0034     inc cx
0035     inc si
0036     jmp strlen_A
0037
0038 strlen_A_done:
0039     mov n, cx
0040
0041 lea si, B
0042 mov cx, 0
0043 strlen_B:
0044     cmp byte ptr [si], '5'
0045     je strlen_B_done
0046     inc cx
0047     inc si
0048     jmp strlen_B
0049
0050 strlen_B_done:
0051     mov m, cx
0052
0053 ;Kiem tra xau B co phai la con cua xau A khong
0054 lea di, 0
0055 mov found, 0

```

```

0090
0091 Output Proc
0092
0093     LEA DX, CRTF           ;hien thi xuong dong
0094     MOV AH, 9
0095     INT 21H
0096
0097     LEA DX, MS2
0098     MOV AH, 9
0099     INT 21H
0100
0101     MOV AX, Cnt           ;AX = sum
0102     MOV BX, 10            ;BX = 10
0103     MOV CX, 0            ;CX = 0 : dem so luong push vao stack
0104
0105     chia:
0106     MOV DX, 0
0107     DIV BX                ;AX/10 phan du luu tai DX
0108     PUSH DX              ;push DX vao stack
0109     INC CX                ;CX++
0110     CMP AX, 0            ;neu AX = 0 thi jump den hienthi
0111     JE hienthi
0112     JMP chia              ;tiếp tục chia
0113
0114     hienthi:
0115     POP DX                ;lay DX ra khoi stack
0116     ADD DL, 48            ;chuyen tu so sang ki tu
0117     MOV AH, 2            ;in ra ki tu vua xu li
0118     INT 21H
0119     DEC CX                ;CX--
0120     CMP CX, 0            ;neu CX = 0 thi dung
0121     JNE hienthi
0122
0123     ret
0124 Output Endp

```

Câu 20:

```
emu8086 - assembler and microprocessor emulator 4.08
file edit bookmarks assembler emulator math ascii codes help
new open examples save compile emulate calculator convertor options help about

0004 .model small
0005 .stack 100h
0006 .data
0007 msg1 db "Hay Nhap Vao Chuoi A: $"
0008 msg2 db 10,13,"So lan xuất hiện của chuỗi 'knt' trong chuỗi A là: $"
0009 str1 db 256 dup('$')
0010
0011 x dw ?
0012
0013 .code
0014
0015 main proc
0016     mov ax,@data
0017     mov ds,ax ;khởi tạo thanh ghi ds
0018
0019     lea dx,msg1 ;thông báo nhập chuỗi
0020     mov ah,9 ;sử dụng ah9 để in ra thông báo
0021     int 21h
0022
0023     ;Nhập chuỗi A
0024     lea dx,str1
0025     mov ah,10 ;sử dụng ah10 để nhập chuỗi A
0026     int 21h
0027
0028     xor cx,cx;
0029     lea si,str1+2; đưa con trỏ đến vị trí địa chỉ của mảng
0030     mov cl,[si]
0031     while:
0032         mov dl,[si] ;lưu ký tự vào dl
0033         cmp dl,'k' ;so sánh ký tự đã lưu ở dl với ký tự 'k'
0034         jne continue ;nếu không thì nhảy đến continue
0035         inc si ;tăng si lên một đơn vị
0036         dec cx ;giảm cx
0037         cmp cx,0 ;so sánh cx với 0;
0038         je print
0039
0040         mov dl,[si] ;lưu ký tự vào dl
0041         cmp dl,'t' ;so sánh ký tự đã lưu ở dl với ký tự 't'
0042         jne continue ;nếu không thì nhảy đến continue
0043         inc si ;tăng si lên một đơn vị
0044         dec cx ;giảm cx
0045         cmp cx,0 ;so sánh cx với 0;
0046         je print
0047
0048         mov dl,[si] ;lưu ký tự vào dl
0049         cmp dl,'n' ;so sánh ký tự đã lưu ở dl với ký tự 'n'
0050         jne continue ;nếu không thì nhảy đến continue
0051         inc si ;tăng si lên một đơn vị
0052         dec cx ;giảm cx
0053         cmp cx,0 ;so sánh cx với 0;
0054
line: 104 col: 23 drag a file here to open
```

```
emu8086 - assembler and microprocessor emulator 4.08
file edit bookmarks assembler emulator math ascii codes help
new open examples save compile emulate calculator convertor options help about

0064
0065     mov dl,[si]
0066     cmp dl,' '
0067     jne continue
0068
0069 count:
0070     mov ax,1 ;khởi tạo ax = 1
0071
0072     add x,ax ; x = x + ax
0073     cmp cx,0
0074     je print
0075
0076 continue:
0077     inc si
0078     dec cx
0079     cmp cx,0
0080     jne while
0081     print:
0082         ; in ra thông báo số lần xuất hiện
0083         mov ah,9
0084         lea dx,msg2
0085         int 21h
0086
0087         ;in ra số lần xuất hiện
0088
0089     mov bx,10; khởi tạo bx = 10
0090     mov ax,x; khởi tạo ax = x
0091     mov cx,0
0092
0093     check1:
0094         mov dx,0; khởi tạo phần dư 0
0095         div bx; chia 10 lấy dư
0096         push dx; đẩy dx vào ngăn xếp
0097         inc cx; tăng cx lên 1 đơn vị
0098         cmp ax,0; so sánh phần thương ≠ 0
0099         jnc check1
0100     check2:
0101         pop dx; lấy dx ra khỏi ngăn xếp
0102         add dx,0; đổi số sang ký tự
0103         mov ah,2; sử dụng ah2 để in ra ký tự
0104         int 21h
0105         loop check2
0106
0107
0108     mov ah,4
0109     int 21h
0110
0111 main endp
0112
0113
line: 101 col: 23 drag a file here to open
```

II. Lập trình điều khiển đèn Led, điều khiển nhiệt kế

1. Chương trình hợp ngữ để 8 LED nối với cổng ra 2C0H sáng rồi tắt led. đèn được bật sáng nếu bit điều khiển tương ứng nhận giá trị 0. Ngược lại khi bit điều khiển bằng 1 thì đèn sẽ tắt:

```
emu8086 - assembler and microprocessor emulator 4.08
file edit bookmarks assembler emulator math ascii codes help
new open examples save compile emulate calculator convertor options help about
01 .Model Small
02 .stack 100
03 .Data
04 DK_LED EQU 2C0H ; cong dieu khien LED
05
06 .Code
07
08 Lap: MOV AL, FFH ; tat tat ca LED
09 OUT DK_LED, AL
10 MOV CX, 64 ; tre 64 lenh NOP
11
12 Tre_1: NOP
13 LOOP Tre_1
14 XOR AL, AL ; Bat tat ca LED
15 OUT DK_LED, AL
16 MOV CX, 64
17
18 Tre_2: NOP
19 LOOP Tre_2 ; tre 64 lenh NOP
20
21 JMP Lap
22
23 end
24
25
26
```

2. Lập trình hợp ngữ điều khiển thiết bị ảo Nhiệt kế và bếp (Therometer & Heater) sử dụng 2 cổng:

- Cổng điều khiển số 127 để nhận byte điều khiển bếp: 0 để tắt bếp, 1 để bật bếp.
- Cổng dữ liệu số 125 để đọc dữ liệu là nhiệt độ đo được bởi nhiệt kế.

```
emu8086 - assembler and microprocessor emulator 4.08
file edit bookmarks assembler emulator math ascii codes help
new open examples save compile emulate calculator convertor options help about
01 .Model Small
02 .stack 100
03 .Data
04
05 .Code
06
07 start:
08     in al, 125 ; doc nhiet do hien tai
09     cmp al, 60 ; so sanh nhiet do hien tai voi 60 do
10     jl low ; neu thap hon, nhay den nhan low
11
12     cmp al, 80 ; so sanh nhiet do hien tai voi 80 do
13     jle ok ; nhay den ok neu <=
14     jg high ; nhay den high neu >
15
16 low:
17     mov al, 1
18     out 127, al ; tang nhiet
19     jmp ok
20
21 high:
22     mov al, 0
23     out 127, al ; giam nhiet (tat)
24
25 ok: jmp start ; lap lai tu dau
26
27
28
29 end
30
31
32
```

Phần 4: Giải mã địa chỉ bộ nhớ

(by Hoàng Anh Vũ)

1. Giải mã địa chỉ bộ nhớ sử dụng mạch logic cơ bản:

(chỉ sử dụng các cổng NOT, NAND, ...)



Các bước xây dựng:

Bước 1 : Xác định số bit cho địa chỉ nội bộ chip và mạch giải mã.

Bước 2 : Phân giải địa chỉ cơ sở của các chip.

Bước 3 : Vẽ sơ đồ bit.

Bước 4 : Vẽ hình mạch giải mã.

Ví dụ : Xây dựng mạch giải mã địa chỉ cho 1 bộ nhớ ROM có dung lượng 4KB bằng phương pháp sử dụng mạch logic cơ bản; Biết rằng kích thước 1 vi mạch nhớ là 2Kx8 và địa chỉ cơ sở là 03800H.

→ Phân tích đề:

+ dung lượng bộ nhớ ROM : **C = 4KB.**

+ kích thước một vi mạch nhớ : **IC = 2Kx8**

+ địa chỉ cơ sở : **DCCS = 03800H**

Bài làm :

Bước 1 : Xác định số bit cho địa chỉ nội bộ chip và mạch giải mã.

Ta có : Chip nhớ IC 2Kx8 chiếm không gian $2KB = x 2^{10} = 2^{11} B$

→ cần 11 bit địa chỉ nội bộ chip (A0 – A10)

Vi xử lý 8086 có 20 bit địa chỉ nên ta có $20 - 11 = 9$

→ cần 9 bit cho mạch giải mã.

Bước 2 : Phân giải địa chỉ cơ sở của các chip.

Ta có:

Bộ nhớ có dung lượng 4KB và mỗi chip nhớ có dung lượng 2KB.

→ cần phải có 4KB : $2KB = 2$ chip nhớ.

Mà $2KB = 2^{11} B = 0000\ 0000\ 1000\ 0000\ 0000\ (B) = 00800\ (H)$.

→ Dung lượng của một chip nhớ là 00800(H).

Phân giải địa chỉ cơ sở: Địa chỉ cuối = Địa chỉ đầu + Dung lượng – 1.

+ Địa chỉ của IC 1: Từ **03800H** đến **(03800H + 00800H – 1H)**

= Từ **03800H** đến (**04000H - 1H**)

= Từ **03800H** đến **03FFFH**.

+ Địa chỉ của IC 2: Từ **04000H** đến **047FFH**.

Bước 3 : Vẽ sơ đồ bit.

		9 bit cao									11 bit địa chỉ nội bộ chip (A ₀ - A ₁₀)										
IC1	03800H	0	0	0	0	0	0	1	1	1	0	0	0	0	0	0	0	0	0	0	0
IC2	04000H	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0
		A ₁₉	A ₁₈	A ₁₇	A ₁₆	A ₁₅	A ₁₄	A ₁₃	A ₁₂	A ₁₁	A ₁₀	A ₉	A ₈	A ₇	A ₆	A ₅	A ₄	A ₃	A ₂	A ₁	A ₀

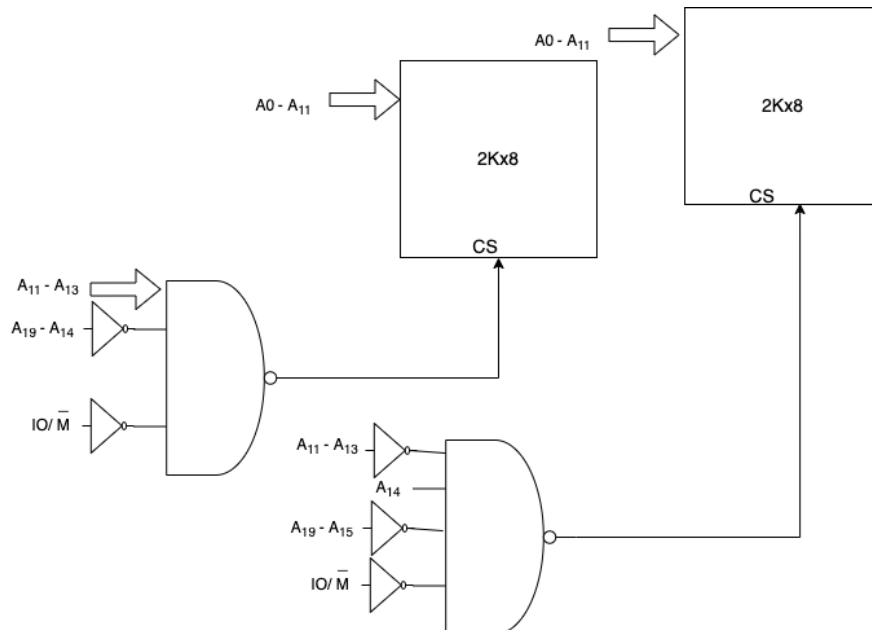
Giải thích :

03800 H = 000000**111**000000000000 B

04000 H = 00000**1**0000000000000000 B

Ta phân biệt 9 bit cao dành cho mạch giải mã (A₁₉ - A₁₀) và 11 bit dành cho địa chỉ nội bộ chip (A₀ - A₁₀).

Bước 4 : Vẽ mạch giải mã.



Giải thích :

+ Các bit 0 được đưa vào qua cổng **NOT** trong khi các bit 1 được truyền thẳng vào cổng NAND.

+ Đầu ra từ cổng NAND đi vào đi vào **IC1** và **IC2** tương ứng như hình vẽ.

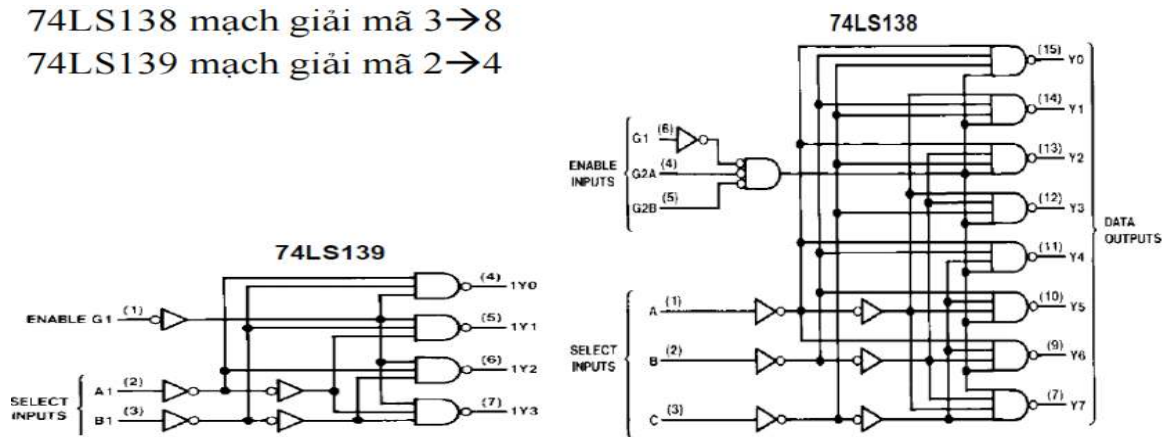
+ IO / : tín hiệu CPU chọn làm việc với thiết bị vào ra hay bộ nhớ. Qua cổng NOT nhận IO / = 0 → CPU chọn làm việc với bộ nhớ.

2. Giải mã địa chỉ bộ nhớ sử dụng mạch tích hợp 74LS138 và 74LS139.

Minh họa về 74LS138 và 74LS139:

74LS138 mạch giải mã $3 \rightarrow 8$

74LS139 mạch giải mã $2 \rightarrow 4$



Ví dụ: Xây dựng bộ giải mã địa chỉ bộ nhớ có dung lượng 8KB có địa chỉ bắt đầu là 0F800H với các chip nhớ có dung lượng 2Kx8. Chỉ được sử dụng các chip giải mã địa chỉ **74LS139** (Là các chip giải mã có 2 đầu vào và 4 đầu ra).

→ Phân tích đề:

- + dung lượng bộ nhớ ROM : **C = 8KB.**
- + kích thước một vi mạch nhớ : **IC = 2Kx8**
- + địa chỉ cơ sở : **DCCS = 0F800H**

Bài làm:

Bước 1 : Xác định số bit cho địa chỉ nội bộ chip và mạch giải mã.

Chip nhớ IC 2Kx8 chiếm không gian $2KB = 2^1 \times 2^{10} = 2^{11}$ B

→ Cần 11 bit địa chỉ nội bộ chip (A0 – A10).

Vì xử lý 8086 có 20 bit địa chỉ nên ta có $20 - 11 = 9$

→ cần 9 bit cho mạch giải mã.

Bước 2 : Phân giải địa chỉ cơ sở của các chip.

Bộ nhớ có dung lượng 8KB và mỗi chip nhớ có dung lượng 2KB → cần $8/2 = 4$ IC.

Mà $2KB = 2^{11} = 0000\ 0000\ 1000\ 0000\ 0000(B) = 00800(H)$ → Dung lượng của một chip nhớ là 00800(H).

Ta có : Địa chỉ cuối = Địa chỉ đầu + Dung lượng – 1. (tính toán tương tự phần 1)

+ Địa chỉ của IC 1: Từ **0F800H** đến **0FFFFH** .

+ Địa chỉ của IC 2: Từ **10000H** đến **107FFH** .

+ Địa chỉ của IC 3: Từ **10800H** đến **10FFFH** .

+ Địa chỉ của IC 4: Từ **11000H** đến **117FFH** .

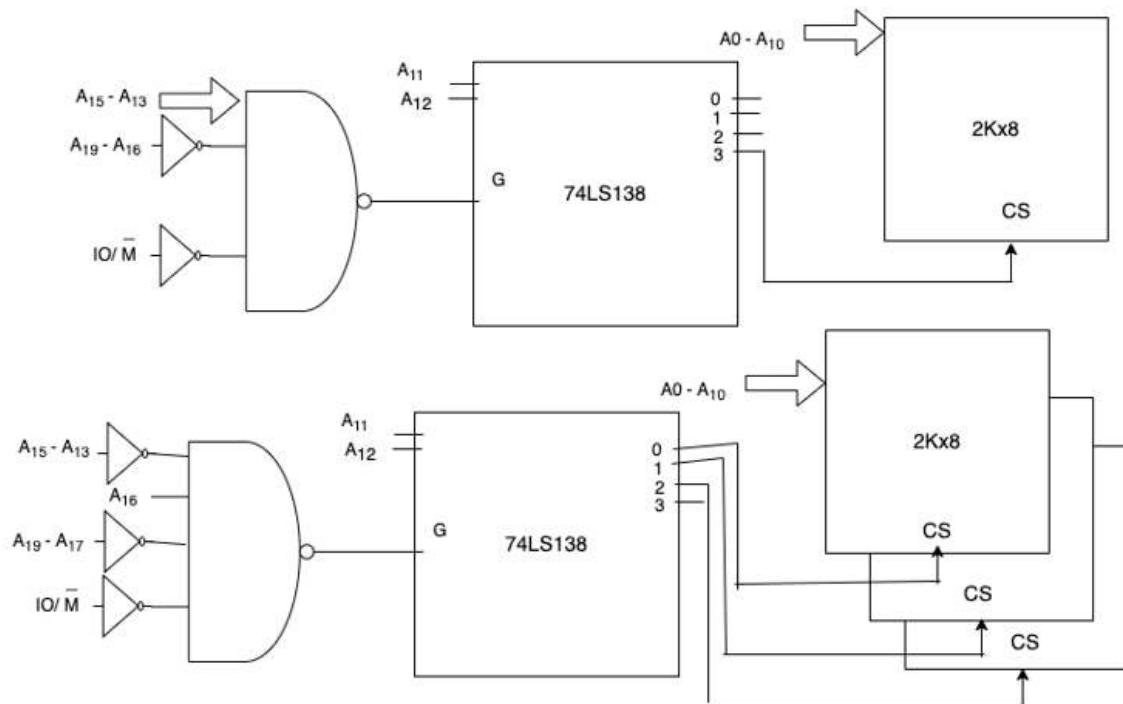
Bước 3 : Vẽ sơ đồ bit.

		7 bit cao							2 bit vào		11 bit địa chỉ nội bộ chip ($A_0 - A_{10}$)										
IC1	0F800H	0	0	0	0	1	1	1	1	1	0	0	0	0	0	0	0	0	0	0	0
IC2	10000H	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
IC3	10800H	0	0	0	1	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0
IC4	11000H	0	0	0	1	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0
		A ₁₉	A ₁₈	A ₁₇	A ₁₆	A ₁₅	A ₁₄	A ₁₃	A ₁₂	A ₁₁	A ₁₀	A ₉	A ₈	A ₇	A ₆	A ₅	A ₄	A ₃	A ₂	A ₁	A ₀

Giải thích :

Ta phân biệt 9 bit cao dành cho mạch giải mã ($A_{19} - A_{10}$), trong đó có 2 bit vào (A_{11} và A_{12}) cho **74LS139** và 11 bit dành cho địa chỉ nội bộ chip ($A_0 - A_{10}$).

Bước 4 : Vẽ mạch giải mã.



Giải thích :

+ Xét 7 bit cao ($A_{13} - A_{19}$) cần 2 chip giải mã 74LS139:

- Một cho 0F800H
- Một cho 10000H, 10800H, 11000H (vì 7 bit giống nhau)

+ Các bit 0 được đưa vào qua cổng **NOT** trong khi các bit 1 được truyền thẳng vào cổng NAND.

+ $IO/\bar{}$: tín hiệu CPU chọn làm việc với thiết bị vào ra hay bộ nhớ. Qua cổng NOT nhận $IO/\bar{}$ = 0 $\rightarrow$ CPU chọn làm việc với bộ nhớ.

+ A11 và A12 dành cho đầu vào của **74LS139**.

Ví dụ : Xây dựng bộ giải mã địa chỉ bộ nhớ có dung lượng 8KB có địa chỉ bắt đầu là 0F800H với các chip nhớ có dung lượng 2Kx8. Chỉ được sử dụng các chip giải mã địa chỉ **74LS138** (Là các chip giải mã có 3 đầu vào và 8 đầu ra).

$\rightarrow$ Phân tích đề:

+ dung lượng bộ nhớ ROM : **C = 8KB**.

+ kích thước một vi mạch nhớ : **IC = 2Kx8**

+ địa chỉ cơ sở : **DCCS = 0F800H**

Bước 1 : Xác định số bit cho địa chỉ nội bộ chip và mạch giải mã.

Chip nhớ IC 2Kx8 chiếm không gian 2KB = $2^1 \times 2^{10} = 2^{11}$ B

$\rightarrow$ Cần 11 bit địa chỉ nội bộ chip (A0 – A10).

Vì xử lý 8086 có 20 bit địa chỉ nên ta có 20 – 11 = 9

$\rightarrow$ cần 9 bit cho mạch giải mã.

Bước 2 : Phân giải địa chỉ cơ sở của các chip.

Bộ nhớ có dung lượng 8KB và mỗi chip nhớ có dung lượng 2KB $\rightarrow$ cần $8/2 = 4$ IC.

Mà 2KB = 211 = 0000 0000 **1**000 0000 0000(B) = 00800(H) $\rightarrow$ Dung lượng của một chip nhớ là 00800(H).

Ta có : Địa chỉ cuối = Địa chỉ đầu + Dung lượng – 1. (tính toán tương tự phần 1)

+ Địa chỉ của IC 1: Từ **0F800H** đến **0FFFFH** .

+ Địa chỉ của IC 2: Từ **10000H** đến **107FFH** .

+ Địa chỉ của IC 3: Từ **10800H** đến **10FFFH** .

+ Địa chỉ của IC 4: Từ **11000H** đến **117FFH** .

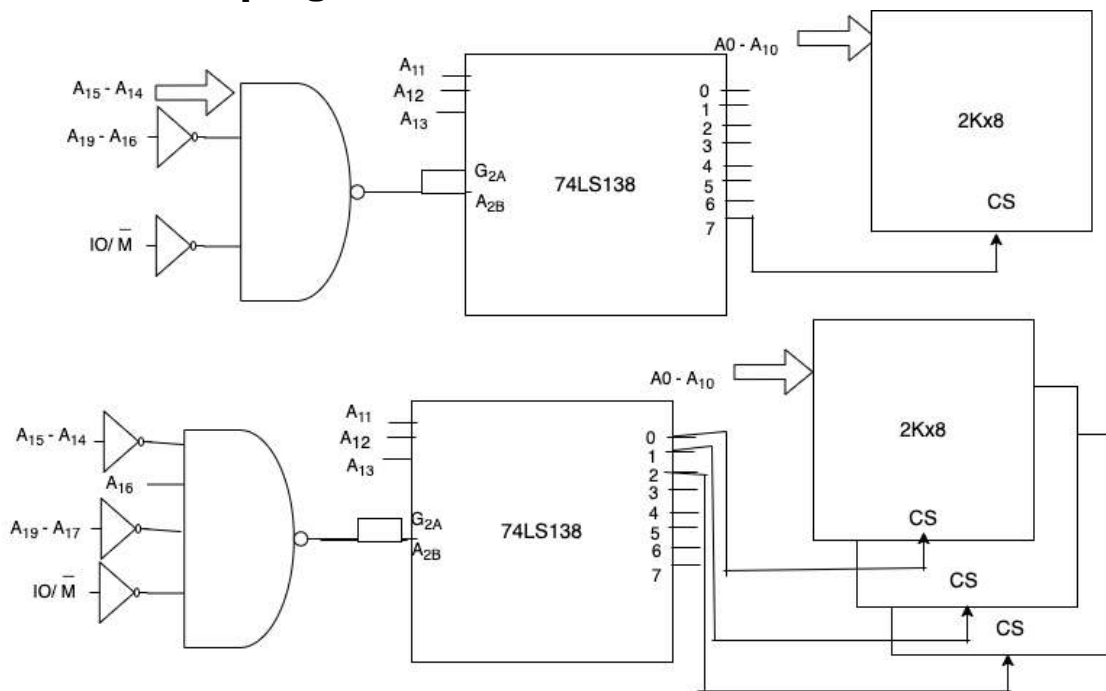
Bước 3 : Vẽ sơ đồ bit.

		6 bit cao						3 bit vào			11 bit địa chỉ nội bộ chip ($A_0 - A_{10}$)										
IC1	0F800H	0	0	0	0	1	1	1	1	1	0	0	0	0	0	0	0	0	0	0	
IC2	10000H	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
IC3	10800H	0	0	0	1	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	
IC4	11000H	0	0	0	1	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	
		A ₁₉	A ₁₈	A ₁₇	A ₁₆	A ₁₅	A ₁₄	A ₁₃	A ₁₂	A ₁₁	A ₁₀	A ₉	A ₈	A ₇	A ₆	A ₅	A ₄	A ₃	A ₂	A ₁	A ₀

Giải thích :

Ta phân biệt 9 bit cao dành cho mạch giải mã (A19 – A10), trong đó có 3 bit vào (A11, A12 và A13) cho **74LS138** và 11 bit dành cho địa chỉ nội bộ chip (A0 – A10).

Bước 4 : Vẽ mạch giải mã.



Giải thích:

Xét 6 bit cao (A14 – A19) cần 2 chip giải mã 74LS138:

- Một cho 0F800H
- Một cho 10000H, 10800H, 11000H (vì 6 bit giống nhau).

+ Các bit 0 được đưa vào qua cổng **NOT** trong khi các bit 1 được truyền thẳng vào cổng NAND.

+ IO / : tín hiệu CPU chọn làm việc với thiết bị vào ra hay bộ nhớ. Qua cổng NOT nhận $IO / = 0 \rightarrow CPU$ chọn làm việc với bộ nhớ.

+ A11, A12 và A13 dành cho đầu vào của **74LS138**.

So sánh ưu điểm và nhược điểm giữa 2 cách xây dựng bộ giải mã địa chỉ bộ nhớ.

	Mạch logic cơ bản	Mạch tích hợp
Ưu điểm	+ Cho phép tạo mạch giải mã đầy đủ. + Tương đối đơn giản rẻ tiền khi chỉ cần 1 hoặc ít đầu ra	+ Cho phép tạo mạch giải mã đầy đủ. + Cho phép tạo mạch giải mã chấp nhận một số hạn chế đầu vào và tạo ra một

		số hạn chế tín hiệu chọn mạch đầu ra.
Nhược điểm	+ Cồng kềnh khi cần giải mã cho nhiều đầu ra do số mạch tăng nhanh.	+ Không thích hợp với mạch giải mã cần chấp nhận một số lượng lớn tín hiệu đầu vào và sinh ra nhiều tín hiệu đầu ra. ☞ Cần sử dụng bổ sung mạch logic phụ thì mạch tích hợp mới có thể cho phép giải mã đầy đủ.